

编译原理

--总复习

刘爽

中国人民大学信息学院

■ Summary

- 词法分析：正则文法、正则表达式、有限自动机
- 语法分析
 - 自上而下：LL(1)文法、预测分析、递归下降
 - 自下而上：LR(0), SLR, LR
- 语法制导翻译/中间代码生成：属性文法
- 优化：控制流、数据流分析；前向分析，后向分析
- 目标代码生成：指令选择、寄存器分配、指令调度

词法分析

■ 正则文法

文法 $G = (V_T, V_N, P, S)$ V_T 是非空有限集, 每个元素是一个终结符。
 V_N 是非空有限集, 每个元素是一个非终结符. S 是一个非终结符, 是开始符号 (S 在产生式的左部必须至少出现一次), P 是产生式的集合。

- 如果 P 中的每一个产生式的形式为

$$A \rightarrow aB \quad \text{或} \quad A \rightarrow a$$

其中, $A, B \in V_N, a \in V_T \cup \{\epsilon\}$, 则称 G 是**右线性文法**。

- 若文法 G 中的每一个产生式的形式为

$$A \rightarrow Ba \quad \text{或} \quad A \rightarrow a, \quad \text{则称 } G \text{ 是左线性文法。}$$

右线性文法和左线性文法都称为正则文法 (3型文法), 其所产生的语言都称为**正则语言** 或3型语言。

■ 正则表达式与正则集

- 正则表达式可以详细说明单词符号的结构，可以精确地定义集合（正则集合）。正则表达式和正则集合的递归定义如下：

1. ε 和 Φ 都是 Σ 上的正则式，其正则集为 $\{\varepsilon\}$ 和 Φ ；
2. 任何 $a \in \Sigma$ ， a 是 Σ 的一个正则式，正则集为 $\{a\}$ ；
3. 假定 U 和 V 都是 Σ 上的正则式，正则集为 $L(U)$ 和 $L(V)$ ，那么，

$(U|V)$, (UV) , $(U)^*$ 也都是 S 上的正则式，正则集为 $L(U) \cup L(V)$, $L(U)L(V)$ 和 $L(U)^*$ ；

仅由有限次使用上述三步骤而定义的表达式才是 Σ 上的正则表达式，仅由这些正则表达式所表示的字集才是 Σ 上的正则集。

正则集合（正规集）：可以用一个正则表达式定义的语言。

有限自动机

- **有限自动机**（也称有穷自动机）作为一种**识别**装置，它能准确地识别**正则集合**，即识别**正则文法所定义的语言与正则表达式所表示的集合**，引入有穷自动机这个理论，正是为词法分析程序的自动构造寻找特殊的方法和工具。
- **有限自动机**分为两类：
 - **确定有限自动机** Deterministic Finite Automata (DFA)
 - **不确定有限自动机** Nondeterministic Finite Automata (NFA)
- 其中“不确定”的含义是：对于**某个输入符号**，在**同一状态**上存在**不止一种转换**。
- 确定和不确定有限自动机都能而且仅能识别**正则集合**，即它们能够识别正则表达式所表示的语言。

■ 确定有限自动机 (DFA)

- 一个确定的有限自动机(DFA)是一个五元式: $M=(S, \Sigma, \delta, s_0, F)$ 其中
 - S 是一个有限集, 它的每个元素称为一个状态。
 - Σ 是一个有穷字母表, 它的每个元素称为一个输入字符
 - $\delta: S \times \Sigma \rightarrow S$ 是一个单值映射 (确定性)。
 - $\delta(s, a)=s'$ 意味着: 当现行状态为 s 、输入字符为 a 时, 将转换到下一个状态 s' 。我们称 s' 为 s 的一个后继状态。
 - $s_0 \in S$, 是唯一的初态。
 - $F \subseteq S$, 是一个终态集 (可空)

■ 非确定有限自动机 (NFA)

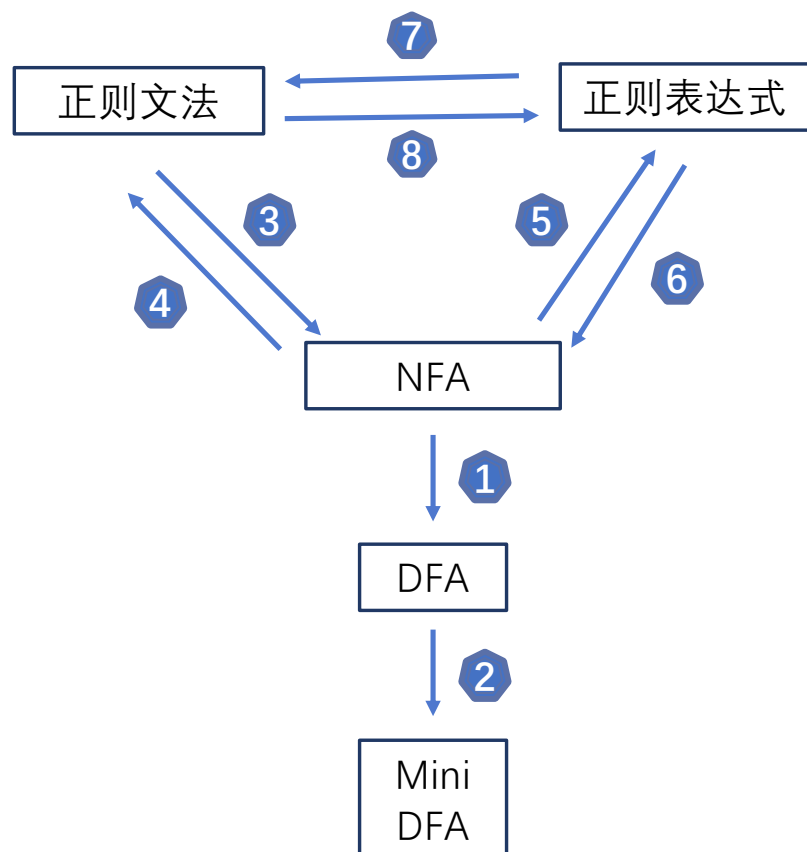
- 一个非确定有限自动机(NFA)是一个五元式: $M=(S, \Sigma, \delta, S_0, F)$

其中

- S 是一个有限集, 它的每个元素称为一个状态。
- Σ 是一个有穷字母表, 它的每个元素称为一个输入字符
- $\delta: S \times \Sigma^* \rightarrow 2^S$ 是一个从 $S \times \Sigma^*$ 至 **S 子集**的单值映射。
- $S_0 \subseteq S$, 是一个**非空的初态集**
- $F \subseteq S$, 是一个终态集 (可空)

若 δ 是一个多值函数, 且输入可允许为 ε , 则有穷自动机是不确定的, 即在某个状态下, 对于某个输入字符存在多个后继状态。

Summarize



1. NFA → DFA (NFA 确定化)
2. DFA → mini DFA (DFA 最小化)
3. 正则文法 → NFA
4. NFA → 正则文法
5. NFA → 正则表达式
6. 正则表达式 → NFA
7. 正则表达式 → 正则文法
8. 正则文法 → 正则表达式

DFA和NFA的等价性

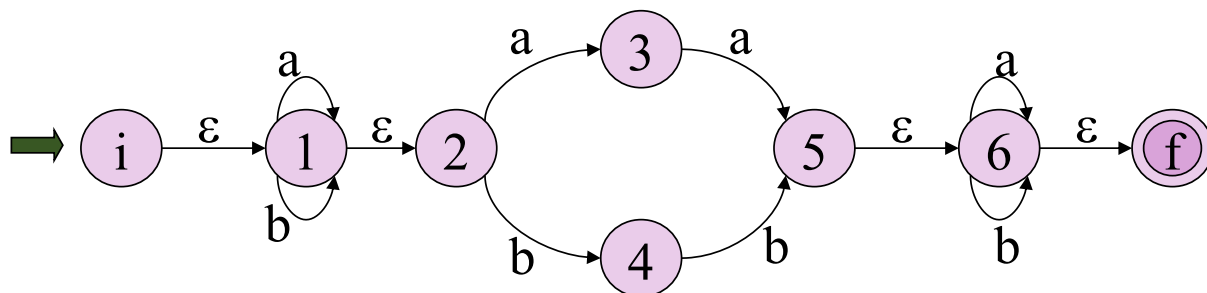
- 对任何一个NFA $M = (S, \Sigma, f, S_0, F)$, 都存在一个DFA $M' = (S', \Sigma, f', s_0, F')$, 使得 $L(M') = L(M)$ 。
 - 证明的思想是由M出发构造等价的M', 办法是让M'的一个状态对应于M的一个状态集合。
 - 即若 $\delta(s, a) = \{s_1, s_2, \dots, s_k\}$,
 - 我们将集合 $\{s_1, s_2, \dots, s_k\}$ 作为一个整体看作M'中的一个状态, 即S'中的一个元素。

■ NFA的确定化

- 显然，DFA是NFA的特例，但是对于每个NFA M 都存在一个DFA M'' ，使 $L(M)=L(M'')$ 。
- 定义对状态集合 I 的几个有关运算：
 - 状态集合 I 的 ϵ -闭包，表示为 $\epsilon\text{-closure}(I)$ ，定义为一状态集，包括
 - (1) 状态集 I 中的任何状态 s 经任意条 ϵ 弧而能到达的状态的集合；
 - (2) 状态集 I 的任何状态 s 都属于 $\epsilon\text{-closure}(I)$ 。
 - 状态集合 I 的 a 弧转换 $I_a = \epsilon\text{-closure}(\text{move}(I, a)) = \epsilon\text{-closure}(J)$ ，其中 $\text{move}(I, a)$ 为状态集合 J ，其定义是所有那些可从 I 的某一状态经过一条 a 弧而到达的状态的全体。

NFA确定化的例子

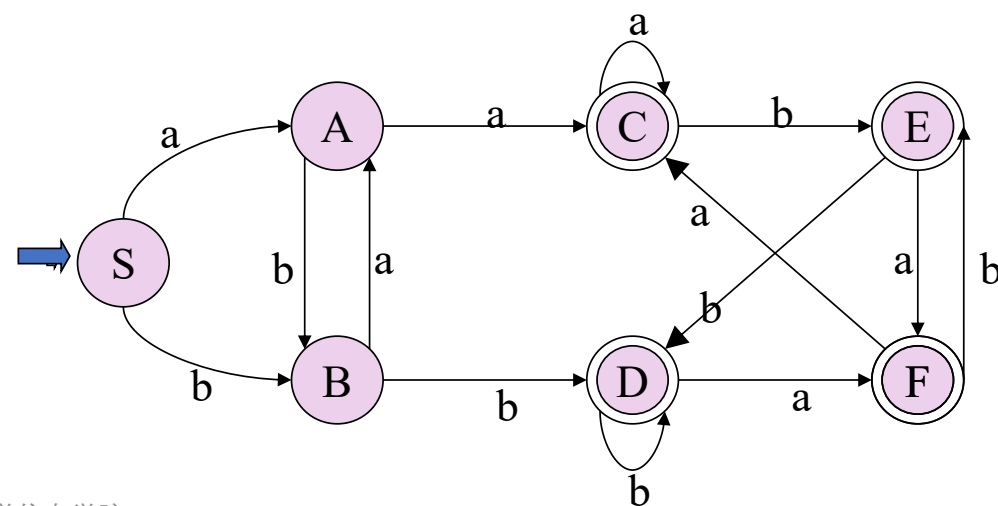
$$I_a = \varepsilon\text{-closure}(\text{move}(I, a)) = \varepsilon\text{-closure}(J)$$



包含原初始状态i的状态子集为
DFA的初态

包含原终止状态f的状态子集为
DFA的终态

	I_a	I_b
$\{i, 1, 2\}$ S	$\{1, 2, 3\}$ A	$\{1, 2, 4\}$ B
$\{1, 2, 3\}$ A	$\{1, 2, 3, 5, 6, f\}$ C	$\{1, 2, 4\}$ B
$\{1, 2, 4\}$ B	$\{1, 2, 3\}$ A	$\{1, 2, 4, 5, 6, f\}$ D
$\{1, 2, 3, 5, 6, f\}$ C	$\{1, 2, 3, 5, 6, f\}$ C	$\{1, 2, 4, 6, f\}$ E
$\{1, 2, 4, 5, 6, f\}$ D	$\{1, 2, 3, 6, f\}$ F	$\{1, 2, 4, 5, 6, f\}$ D
$\{1, 2, 4, 6, f\}$ E	$\{1, 2, 3, 6, f\}$ F	$\{1, 2, 4, 5, 6, f\}$ D
$\{1, 2, 3, 6, f\}$ F	$\{1, 2, 3, 5, 6, f\}$ C	$\{1, 2, 4, 6, f\}$ E



DFA的最小化

- DFA的最小化(DFA的化简): 一个有穷自动机通过消除多余状态和合并等价状态而转换成一个最小的与之等价的有穷自动机。
 - 寻找一个状态数比M少的 DFA M' , 使得 $L(M) = L(M')$
- 最小状态DFA
 - 不存在多余状态(死状态)
 - 不存在互相等价 (不可区别) 的两个状态
- 两个状态s和t等价: 从s出发可以读出某个字符串w而到达终态, 从t出发也可以读出字符串w而到达终态; 反之亦然。
- 两个状态s和t可区别: 不满足
 - 兼容性 (一致性) ——同是终态或同是非终态
 - 传播性 (蔓延性) ——从状态s出发读入某个a ($a \in \Sigma$)和从状态t出发读入某个a到达的状态等价。

■ DFA的最小化—算法

- 算法的核心
 - 把M的状态集K分成不相交的子集。
- 结论
 - 接受L的最小状态有穷自动机**不计同构**是唯一的。
- DFA $M = (S, \Sigma, \delta, s_0, s_t)$, 最小状态DFA M'
 - 1.构造状态的初始划分 Π : 终态 s_t 和非终态 $S - s_t$ 两组
 - 2.对 Π 施用传播性原则构造新划分 Π_{new}
 3. 如 $\Pi_{new} == \Pi$,则令 $\Pi_{final} = \Pi$ 并继续步骤4, 否则 $\Pi := \Pi_{new}$ 重复2
 - 4.为 Π_{final} 中的每一组选一代表, 这些代表构成 M' 的状态。若 s 是一代表,且 $\delta(s,a)=t$, 令 r 是 t 组的代表, 则 M' 中有一转换 $\delta'(s,a)=r$ 。 M' 的开始状态是含有 s_0 的那组的代表, M' 的终态是含有 s_t 的那组的代表
 - 5.去掉 M' 中的死状态

■ DFA的最小化—算法 (cont)

- 对 Π 施用传播性原则构造新划分 Π_{new} 步骤：
 - 假设 Π 被分成 m 个子集 $\Pi = \{S_1, S_2, \dots, S_m\}$, 且属于不同子集的状态是可去别的, 检查 Π 的每一个子集 S_i , 看是否能够进一步划分。
 - 对于某个 S_i , 令 $S_i = \{s_1, s_2, \dots, s_k\}$, 若存在数据字符 a 使得 I_a 不全包含在现行 Π 的某一子集 S_j 中, 则将 S_i 一分为二: 即假定状态 s_1, s_2 , 经过 a 弧分别达到状态 t_1, t_2 , 且 t_1, t_2 属于现行 Π 的两个不同子集, 那么将 S_i 分成两半, 一半含有 s_1 : $S_{i1} = \{s \mid s \in S_i \text{ 且 } s \text{ 经 } a \text{ 弧到达 } t_1 \text{ 所在子集种的某状态}\}$; 另一半含有 s_2 : $S_{i2} = S_i - S_{i1}$
 - 由于 t_1, t_2 是可区别的, 即存在 w , 使得 t_1 读出 w 而停于终态, t_2 读不出 w 或读出 w 却未停于终态。因而 aw 可以将 s_1, s_2 区分开。也就是说 S_{i1} 和 S_{i2} 中的状态时可区别的。
 - $\Pi_{\text{new}} = \{S_1, S_2, \dots, S_{i1}, S_{i2}, \dots, S_m\}$

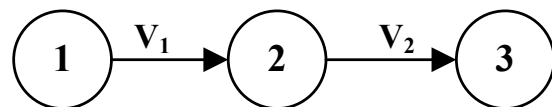
正则表达式和有限自动机的等价性

--有限自动机→正则表达式

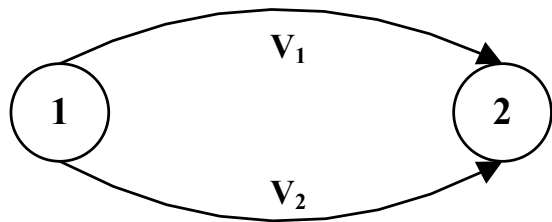
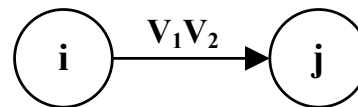
- 关于正则表达式与自动机的等价性，有如下性质：
 - 对任何FA M ，都存在一个正则表达式 r ，使得 $L(r)=L(M)$ 。
 - 对任何正则表达式 r ，都存在一个FA M ，使得 $L(M)=L(r)$ 。

步骤：

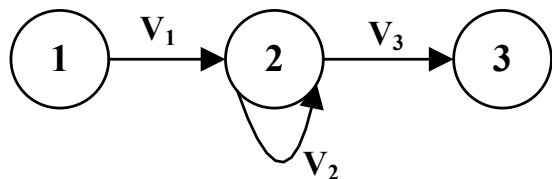
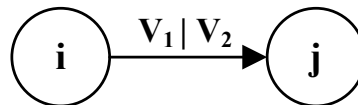
把转换图的意义拓宽，令每条弧上可以标记正则式。



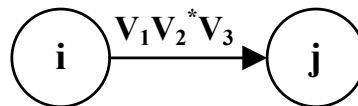
代之以



代之以



代之以



1. 在给定的NFA M 上加两个新结，一个为初态 X ，从 X 用 ϵ 弧连接 M 的所有初态，另一为 Y ，从 M 的所有终态用 ϵ 弧连到 Y ，新的NFA M' 与 M 等价。

2. 对 M' 用左侧等价规则，消除 X, Y 以外的其它结。

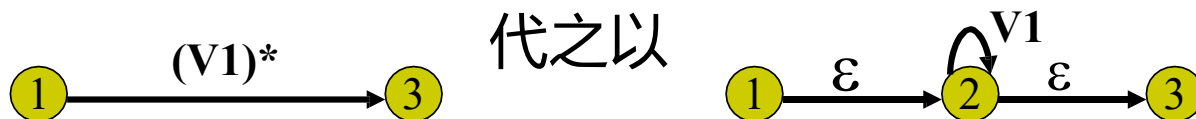
正则表达式和有限自动机的等价性

--正则表达式 $\rightarrow$ 有限自动机(I)

1. 对给定的正则表达式构成一个NFA M 。先写出：



2. 用以下规则对 V 进行分解并加进新节点



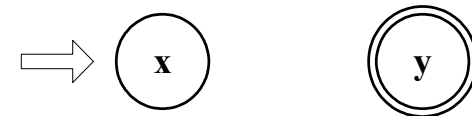
在分解过程中，要求：

- (1) X, Y 始终为唯一的初态和终态。
- (2) 所加新结点其名字彼此不同。
- (3) 弧上的标记必须是字符或空字。

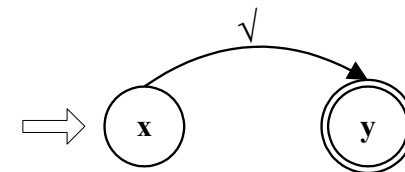
正则表达式和有限自动机的等价性

--正则表达式 $\rightarrow$ 有限自动机(II)

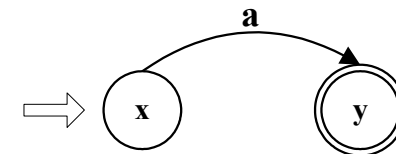
- 对于简单的正则表达式:
 - 对于正则表达式 ϕ , 所构造的NFA为:



- 对于正则表达式 ϵ , 构造的NFA为:



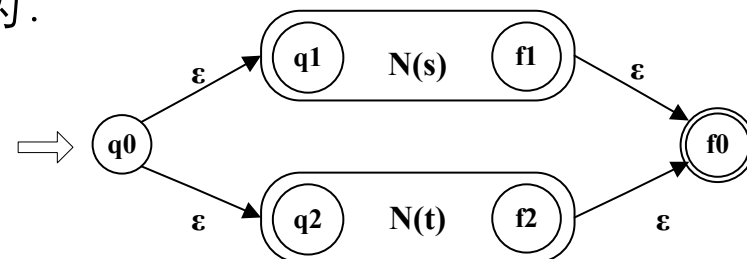
- 对于正则表达式 a , $a \in \Sigma$, 构造的NFA为:



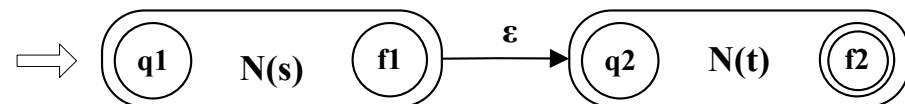
正则表达式和有限自动机的等价性

--正则表达式 $\rightarrow$ 有限自动机 (III)

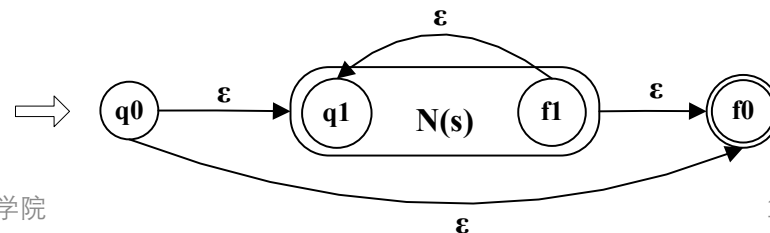
- 若 s, t 为 Σ 上的正则表达式, 相应的NFA分别为 $N(s)$ 和 $N(t)$, 则
 - 对于正则表达式 $R = s \mid t$, 所构造的NFA(R)为:



- 对于正则表达式 $R = st$, 构造的NFA(R)为:



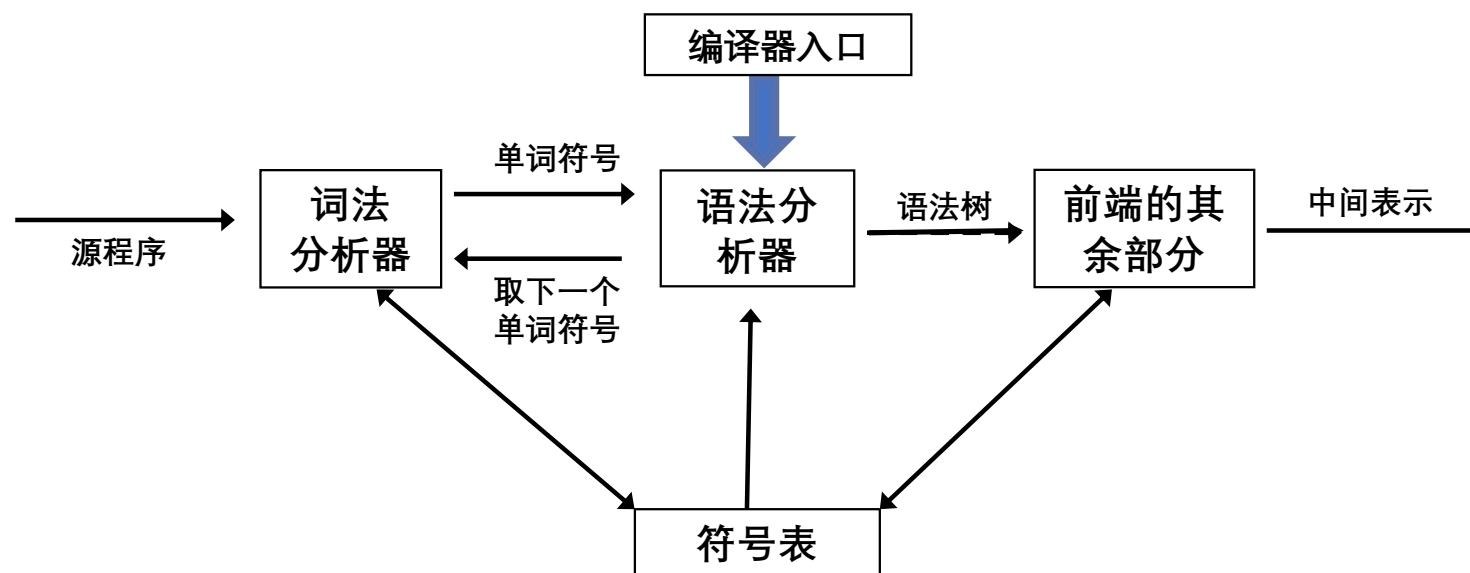
- 对于正则表达式 $R = s^*$, 构造的NFA(R)为:



语法分析

■ 语法分析器的作用

- 接收词法分析器提供的记号串
- 检查记号串是否能由源程序语言的文法产生
- 用易于理解的方式提示语法错误信息，并能从常见的错误中恢复过来



自上而下的语法分析

■ LL(1)分析条件

唯一确定
一个产生式

- 满足构造不带回溯的自顶向下分析的文法条件。
 - 文法不含左递归
 - 对于文法中每一个非终结符A的各个产生式的候选首符集两两不相交。
即, 若 $A \rightarrow \alpha_1 \mid \alpha_2 \mid \cdots \mid \alpha_n$, 则 $\text{FIRST}(\alpha_i) \cap \text{FIRST}(\alpha_j) = \emptyset \quad (i \neq j)$
 - 对文法中的每个非终结符A, 若它存在某个候选首符集包含 ε , 则,
 $\text{FIRST}(A) \cap \text{FOLLOW}(A) = \emptyset$

如果一个文法G满足以上条件, 则称该文法G为LL(1)文法。

这里LL(1)中的第一个L表示从左到右扫描输入串, 第二个L表示最左推导, 1表示分析时每一步只需向前查看一个符号。

■ LL(1)分析条件

- 对于一个LL(1)文法，可以对其输入串进行有效的无回溯的自顶向下分析。
- 假设要用非终结符A进行匹配，面临的输入符号为a，A的所有产生式为 $A \rightarrow \alpha_1 \mid \alpha_2 \mid \cdots \mid \alpha_n$
 - 若 $a \in \text{FIRST}(\alpha_i)$ ，则指派 α_i 去执行匹配任务。
 - 若a不属于任何一个候选首字符集，则：
 - 若 ϵ 属于某个 $\text{FIRST}(\alpha_i)$ ，且 $a \in \text{FOLLOW}(A)$ ，则让A与 ϵ 自动匹配；
 - 否则，a的出现是一种语法错误。

根据LL(1)文法的条件，每一步这样的工作都是确信无疑的

FIRST集合与FOLLOW集合

- 令 G 是一个不含左递归的文法，对 G 的所有非终结符的每个候选 α 定义它的终结首符集 $FIRST(\alpha)$ 为：
 - $FIRST(\alpha) = \{a \mid \alpha \xRightarrow{*} a\cdots, a \in V_T\}$
 - 若 $\alpha \xRightarrow{*} \varepsilon$ ，则规定 $\varepsilon \in FIRST(\alpha)$
- 假定 S 是文法 G 的开始符号，对于 G 的任何非终结符 A ，我们定义它的后随符号集 $FOLLOW(A)$ 为：
 - $FOLLOW(A) = \{a \mid S \xRightarrow{*} \cdots Aa\cdots, a \in V_T\}$
 - 若 $S \xRightarrow{*} \cdots A$ ，则规定 $\# \in FOLLOW(A)$

■ 消除直接左递归 I

$$P \rightarrow P\alpha \mid \beta \quad \xrightarrow{\beta \text{ 不以 } P \text{ 开头}} \quad \begin{aligned} P &\rightarrow \beta P' \\ P' &\rightarrow \alpha P' \mid \varepsilon \end{aligned}$$

得到的非直接左递归形式的文法与原文法等价

例子

$$E \rightarrow E+T \mid T \quad \xrightarrow{\quad} \quad \begin{aligned} E &\rightarrow TE' \\ E' &\rightarrow +TE' \mid \varepsilon \end{aligned}$$

$$T \rightarrow T*F \mid F \quad \xrightarrow{\quad} \quad \begin{aligned} T &\rightarrow FT' \\ T' &\rightarrow *FT' \mid \varepsilon \end{aligned}$$

$$F \rightarrow (E) \mid i \quad \xrightarrow{\quad} \quad F \rightarrow (E) \mid i$$

■ 消除直接左递归 II

- 一般而言,假定关于**A**的全部产生式是:

$$\mathbf{A} \rightarrow \mathbf{A}\alpha_1 | \mathbf{A}\alpha_2 | \dots | \mathbf{A}\alpha_m | \beta_1 | \beta_2 | \dots | \beta_n$$

- 其中 $\beta_i (i=1,2,\dots,n)$ 不以**A**打头;
 $\alpha_j (j=1,2,\dots,m)$ 不等于 ε ,

- 那么可以把上述产生式改写为:

$$\mathbf{A} \rightarrow \beta_1 \mathbf{A}' | \beta_2 \mathbf{A}' | \dots | \beta_n \mathbf{A}'$$

$$\mathbf{A}' \rightarrow \alpha_1 \mathbf{A}' | \alpha_2 \mathbf{A}' | \dots | \alpha_m \mathbf{A}' | \varepsilon$$

使用该方法,可以把直接左递归都去掉(改为直接右递归)。但这并不意味着已经消除了整个文法的左递归性。

例如文法:

$$S \rightarrow Aa | b$$

$$A \rightarrow Sd | \varepsilon$$

非终结符号S是左递归的,
因为有 $S \Rightarrow Aa \Rightarrow Sda$,
但它不是直接左递归的

■ 消除左递归的一般算法

- 如果一个文法不含回路（形如 $P \xRightarrow{+} P$ 的推导），也不含以 ε 为右部的产生式，那么执行下述算法将保证消除左递归（但改写后的文法可能含有 ε 为右部的产生式）。

消除左递归算法

1.排序

(1) 把文法G的所有非终结符按任意一种顺序排列成 $P_1, P_2, \dots, P_n$; 按此顺序执行

2.1代入

(2) FOR $i:=1$ TO n DO

FOR $j:=1$ TO $i-1$ DO

把形如 $P_i \rightarrow P_j \gamma$ 的规则改写成

$P_i \rightarrow \delta_1 \gamma \mid \delta_2 \gamma \mid \dots \mid \delta_k \gamma$ 其中 $P_j \rightarrow \delta_1 \mid \delta_2 \mid \dots \mid \delta_k$ 是关于 P_j 的所有规则;

消除关于 P_i 规则的直接左递归

2.2 消除直接左递归

3.化简

(3) 化简由(2)得到的文法，即去除那些由开始符号出发永远无法到达的非终结符的产生规则。

■ 消除回溯、提取左因子 I

- 语句 $\rightarrow$ if 条件 then 语句 else 语句 α_i
| if 条件 then 语句 α_j

非终结符“语句”的两个候选的终结首符集都是 $\{\}$
即 $\text{FIRST}(\alpha_i) \cap \text{FIRST}(\alpha_j) \neq \Phi$

要消除回溯，就要保证对文法G每个非终结符A的所有产生式 $\alpha_i, \alpha_j, i, j \in [1, n]$ 且 $i \neq j$, $\text{FIRST}(\alpha_i) \cap \text{FIRST}(\alpha_j) = \Phi$

如何把一个文法改造成任何非终结符的所有候选首符集两两不相交?
 $\rightarrow$ 提取公共左因子

■ 消除回溯、提取左因子 II

- 提取左因子的方法

- 假定 A 的规则是:

- $A \rightarrow \delta\beta_1 | \delta\beta_2 | \cdots | \delta\beta_n | \gamma_1 | \gamma_2 | \cdots | \gamma_m$ (其中, 每个 γ 不以 δ 开头)

- 那么这些规则可以改写为:

- $A \rightarrow \delta A' | \gamma_1 | \gamma_2 | \cdots | \gamma_m$

- $A' \rightarrow \beta_1 | \beta_2 | \cdots | \beta_n$

- 经过反复提取左因子, 就能够把每个非终结符 (包括新引进者) 的所有候选首符集变成两两不相交。我们为此要付出的代价是大量引进新的非终结符和 ϵ 产生式。

■ LL(1)分析条件

- 如果A的某个候选首符集中包含 ϵ 怎么办?
- 假定S是文法G的开始符号, 对于G的任何非终结符A, 我们定义(后随符号集 FOLLOW)
 - $\text{FOLLOW}(A) = \{a \mid S \xRightarrow{*} \cdots Aa \cdots, a \in V_T\}$
 - 若 $S \xRightarrow{*} \cdots A$, 则规定 $\# \in \text{FOLLOW}(A)$

即 $\text{FOLLOW}(A)$ 是所有句型中出现在紧接A之后的终结符或“#”。

- 开始符号的FOLLOW集初始化时加入“#”。
- 当非终结符A面临输入符号a, 且a不属于A的任意候选首符集但A的某个候选首符集包含 ϵ 时, 只有当 $a \in \text{FOLLOW}(A)$ 时, 才可能允许A自动匹配 ϵ 。

■ 构造FIRST集合的算法 I

对每一个文法符号 $X \in V_T \cup V_N$ 构造 $\text{FIRST}(X)$: 应用下列规则,直到每个集合 FIRST 不再增大为止.

(1)如果 $X \in V_T$,则 $\text{FIRST}(X) = \{X\}$.

(2)如果 $X \in V_N$,且有产生式 $X \rightarrow a \dots$,则把 a 加入到 $\text{FIRST}(X)$ 中;若 $X \rightarrow \varepsilon$ 也是一个产生式,则把 ε 加入到 $\text{FIRST}(X)$ 中.

(3)如果 $X \rightarrow Y \dots$ 是一个产生式且 $Y \in V_N$,则把 $\text{FIRST}(Y) \setminus \{\varepsilon\}$ 加到 $\text{FIRST}(X)$ 中;

如果 $X \rightarrow Y_1 Y_2 \dots Y_k$ 是一个产生式, $Y_1, \dots, Y_{i-1} \in V_N$,而且对任何 $j, j \in [1, i-1]$, $\varepsilon \in \text{FIRST}(Y_j)$, (即 $Y_1 Y_2 \dots Y_{i-1} \xRightarrow{*} \varepsilon$), 则把 $\text{FIRST}(Y_i) \setminus \{\varepsilon\}$ 加到 $\text{FIRST}(X)$ 中; 特别是, 若所有的 $\text{FIRST}(Y_j)$ 均含有 ε , $j=1, 2, \dots, k$, 则把 ε 加到 $\text{FIRST}(X)$ 中.

■ 构造FIRST集合的算法II

对文法G的任何符号串 $\alpha = X_1X_2 \dots X_n$ 构造集合**FIRST(α)**

(1) 首先置 $\text{FIRST}(\alpha) = \text{FIRST}(X_1) \setminus \{\varepsilon\}$.

(2) 如果对任何 $j, j \in [1, i-1], \varepsilon \in \text{FIRST}(X_j)$, 则把 $\text{FIRST}(X_i) \setminus \{\varepsilon\}$ 加入到 $\text{FIRST}(\alpha)$ 中. 特别是, 若所有的 $\text{FIRST}(X_j)$ 均含有 $\varepsilon, j=1, 2, \dots, n$, 则把 ε 加到 $\text{FIRST}(\alpha)$ 中.

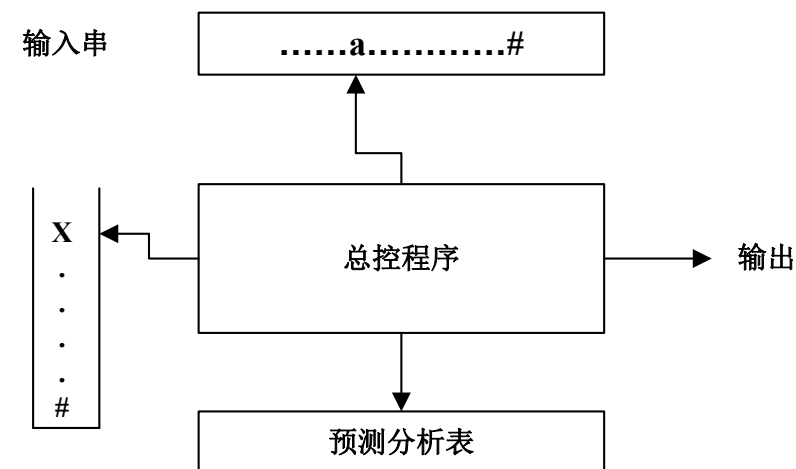
■ 构造结合FOLLOW的算法

对文法G的每个非终结符A构造FOLLOW(A)的办法是：连续应用下列规则,直到每个后随符号集FOLLOW不再增大为止.

- 1) 对于文法的开始符号S, 置#于FOLLOW(S)中;
- 2) 若 $A \rightarrow \alpha B \beta$ 是一个产生式, 则把 $\text{FIRST}(\beta) \setminus \{\epsilon\}$ 加至FOLLOW(B)中;
- 3) 若 $A \rightarrow \alpha B$ 是一个产生式, 或 $A \rightarrow \alpha B \beta$ 是一个产生式而 $\beta \Rightarrow \epsilon$ (即 $\epsilon \in \text{FIRST}(\beta)$), 则把FOLLOW(A)加至FOLLOW(B)中.

■ 预测分析法工作过程概述

- 预测分析程序的总控程序在任何时候都是按STACK栈顶符号X和当前的输入符号a行事的。如下图所示，对于任何(X,a)，总控程序每次都执行下述三种可能的动作之一：
 - 若 $X = a = \text{'\#'}$ ，则宣布分析成功，停止分析过程。
 - 若 $X = a \neq \text{'\#'}$ ，则把X从STACK栈顶弹出，让a指向下一个输入符号。
 - 若X是一个非终结符，则查看分析表M。
 若 $M[X,a]$ 中存放着关于X的一个产生式，那么，先把X弹出STACK栈顶，然后把产生式的右部符号串按**反序**——推进STACK栈（若右部符号为 ϵ ，则意味着不推任何符号进栈）。在把产生式的右部符号推进栈的同时应该做这个产生式对应的语义动作。若 $M[X,a]$ 中存放着“出错标志”，则调用出错诊断程序ERROR。



■ 预测分析算法 I

- 输入:串 w ,文法 G 的分析表 M .
- 输出:如果 $w \in L(G)$,则产生 w 的最左推导,否则输出错误信息.
- 方法: 初态时,分析栈为 $\#S$,栈顶 S 是文法的开始符号;缓冲区为 $w\#$.
分析器按下列操作进行 语法分析:

预测分析算法二

```
(1) push #S; 指针ip指向串w#的第一个符号a;  
(2) repeat  
(3)   令X为栈顶符号,a为ip所指的符号;  
(4)   if X 为终结符或# then  
(5)     if X=a=# then 接受w  
(6)     else if X=a ≠ # then 弹出X,并使ip前进  
(7)     else error  
(8)   else /* X为非终结符*/  
(9)     if M[X,a]=X→Y1Y2...Yk then  
(10)       begin  
(11)         弹出X;  
(12)         push Yk,...,Y2,Y1 /* Y1在栈顶*/  
(13)       end  
(14)     else error  
(15) until X=#; /* 栈为空 */
```

■ 分析表的构造算法

• 构造分析表M的算法如下：

- (1) 对文法G的每个产生式 $A \rightarrow \alpha$, 执行第(2)和(3)步;
- (2) 对每个终结符 $a \in \text{FIRST}(\alpha)$, 把 $A \rightarrow \alpha$ 加入 $M[A, a]$ 中;
- (3) 若 $\varepsilon \in \text{FIRST}(\alpha)$, 则对任何 $b \in \text{FOLLOW}(A)$, 把 $A \rightarrow \alpha$ 加入 $M[A, b]$ 中;
- (4) 把所有无定义的 $M[A, a]$ 标上“出错标志”。

用上述算法可以对任何文法G构造它的分析表M。但对于某些文法, 有些 $M[A, a]$ 可能会有若干个产生式, 或者说有些 $M[A, a]$ 是多重定义的。

如果G是左递归或二义的, 那么M至少含有一个多重定义入口。因此消除左递归和提取左公因子有助于获得无多重定义的分析表

一个文法G的预测分析表M不含多重入口, 当且仅当该文法为LL(1)的

■ 练习

- 考虑文法 $G_1[E]$:

$$E \rightarrow TE'$$
$$E' \rightarrow +E \mid \varepsilon$$
$$T \rightarrow FT'$$
$$T' \rightarrow T \mid \varepsilon$$
$$F \rightarrow PF'$$
$$F' \rightarrow *F' \mid \varepsilon$$
$$P \rightarrow (E) \mid a \mid b \mid \&$$
$$\text{FIRST}(E) = \text{FIRST}(T) = \text{FIRST}(F) = \text{FIRST}(P) = \{ (, a, b, \& \}.$$
$$\text{FIRST}(E') = \{ +, \varepsilon \}$$
$$\text{FIRST}(T') = \{ (, a, b, \&, \varepsilon \}$$
$$\text{FIRST}(F') = \{ *, \varepsilon \}$$
$$\text{FOLLOW}(E) = \text{FOLLOW}(E') \cup \{), \# \} = \{), \# \}$$
$$\text{FOLLOW}(T) = \text{FOLLOW}(T') \cup \text{First}(E') \setminus \{ \varepsilon \} \cup \text{Follow}(E) = \{ +,), \# \}$$
$$\text{FOLLOW}(F) = \text{FOLLOW}(F') \cup \text{First}(T') \setminus \{ \varepsilon \} \cup \text{Follow}(T) = \{ (, a, b, \&, +,), \# \}$$
$$\text{FOLLOW}(P) = \{), a, b, \& \} \cup \text{First}(F') \setminus \{ \varepsilon \} \cup \text{Follow}(F) = \{ *, (, a, b, \&, +,), \# \}$$

- (1) 计算该文法每个非终结符的FIRST和FOLLOW
- (2) 证明这个文法是LL(1)的
- (3) 构造该文法的预测分析表
- (4) 构造该文法的递归下降程序

(2) 证明这个文法是LL(1)的

LL(1) 文法要满足以下三个条件:

(a) 文法不含左递归 满足

(b) 对文法的每一个非终结符A的所有产生式, 其候选首符集两两不相交

只需考察 E', F', F', P 的所有产生式的候选首符集合 满足

(c) 对文法的每一个非终结符A的某个产生式, 若存在某个候选首符集包含 ε , 则

$\text{FIRST}(A) \cap \text{FOLLOW}(A) = \emptyset$, 考察 E', T', F' 满足

故而该文法是LL(1)的

(3) 构造该文法的预测分析表

$\text{FIRST}(TE') = \text{FIRST}(T) = \{ (, a, b, \& \}$

$\text{FIRST}(+E) = \{ + \}$

$\text{FIRST}(FT') = \text{FIRST}(F) = \{ (, a, b, \& \}$

$\text{FIRST}(PF') = \text{FIRST}(P) = \{ (, a, b, \& \}$

$\text{FIRST}(*F') = \{ * \}$

$\text{FIRST}((E)) = \{ \}$

	+	*	(	)	a	b	&	#
E			$E \rightarrow TE'$		$E \rightarrow TE'$	$E \rightarrow TE'$	$E \rightarrow TE'$	
E'	$E' \rightarrow +E$			$E' \rightarrow \epsilon$				$E' \rightarrow \epsilon$
T			$T \rightarrow FT'$		$T \rightarrow FT'$	$T \rightarrow FT'$	$T \rightarrow FT'$	
T'	$T' \rightarrow \epsilon$		$T' \rightarrow T$	$T' \rightarrow \epsilon$	$T' \rightarrow T$	$T' \rightarrow T$	$T' \rightarrow T$	$T' \rightarrow \epsilon$
F			$F \rightarrow PF'$		$F \rightarrow PF'$	$F \rightarrow PF'$	$F \rightarrow PF'$	
F'	$F' \rightarrow \epsilon$	$F' \rightarrow *F'$	$F' \rightarrow \epsilon$	$F' \rightarrow \epsilon$	$F' \rightarrow \epsilon$	$F' \rightarrow \epsilon$	$F' \rightarrow \epsilon$	$F' \rightarrow \epsilon$
P			$P \rightarrow (E)$		$P \rightarrow a$	$P \rightarrow b$	$P \rightarrow \&$	

■ 给出符号串 $a+b$ 的预测分析过程

步骤	符号栈	输入串	所用产生式
1	#E	a+b#	
2	#E'T	a+b#	$E \rightarrow TE'$
3	#)E'T'F	a+b#	$T \rightarrow FT'$
4	#E'T'F'P	a+b#	$F \rightarrow PF'$
5	#E'T'F'a	a+b#	$P \rightarrow a$
6	#E'T'F'	+b#	
7	#E'T'	+b#	$F' \rightarrow \varepsilon$
8	#E'	+b#	$T' \rightarrow \varepsilon$
9	#E+	+b#	$E' \rightarrow +E'$
10	#E	b#	
11	#E'T	b#	$E \rightarrow TE'$
12	#E'T'F	b#	$T \rightarrow FT'$
13	#E'T'F'P	b#	$F \rightarrow PF'$
14	#E'T'F'b	b#	$P \rightarrow b$
15	#E'T'F'	#	
16	#E'T'	#	$F' \rightarrow \varepsilon$
17	#E'	#	$T' \rightarrow \varepsilon$
18	#	#	$E' \rightarrow \varepsilon$
19	#	#	ACC

■ 递归下降分析程序的构造

- 当文法满足**LL(1)**条件时，我们就可以为它构造一个**不带回溯**的**自顶向下**分析程序，这个分析程序是由一组递归过程组成的。每个过程对应文法的一个**非终结符**。这样的一个分析程序称为**递归下降分析器**。

LL(1) 条件

- 文法不含左递归
- 对于文法中每一个非终结符A的各个产生式的候选首符集两两不相交。即，若 $A \rightarrow \alpha_1 \mid \alpha_2 \mid \cdots \mid \alpha_n$ ，则 $\text{FIRST}(\alpha_i) \cap \text{FIRST}(\alpha_j) = \emptyset$ ($i \neq j$)
- 对文法中的每个非终结符A，若它存在某个候选首符集包含 ε ，则， $\text{FIRST}(A) \cap \text{FOLLOW}(A) = \emptyset$

■ 扩充的巴科斯范式—例子

巴科斯范式

$E \rightarrow E + T \mid T$

$T \rightarrow T * F \mid F$

$F \rightarrow (E) \mid i$

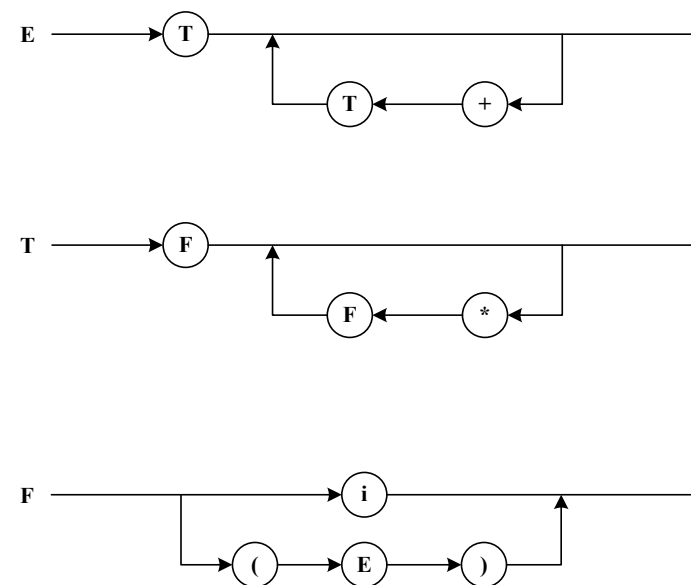
扩充的巴科斯范式

$E \rightarrow T \{ + T \}$

$T \rightarrow F \{ * F \}$

$F \rightarrow (E) \mid i$

语法图



递归下降分析程序构造

$E \rightarrow T\{+T\}$

```
PROCEDURE E;  
BEGIN  
  T;  
  WHILE SYM = '+' DO  
  BEGIN  
    ADVANCE; T  
  END  
END
```

$T \rightarrow F\{*F\}$

```
PROCEDURE T;  
BEGIN  
  F;  
  WHILE SYM = '*' DO  
  BEGIN  
    ADVANCE; F  
  END  
END
```

$F \rightarrow (E) \mid i$

```
PROCEDURE F;  
  IF SYM = 'i' THEN ADVANCE;  
  ELSE  
    IF SYM = '(' THEN  
      BEGIN  
        ADVANCE  
        E;  
        IF SYM = ')' THEN ADVANCE;  
        ELSE ERROR;  
      END  
    ELSE ERROR;
```

自下而上的语法分析

■ 短语、直接短语、句柄的定义：

- 文法 $G[S]$, $\alpha\beta\delta$ 是文法 G 的一个 **句型**, $S \overset{*}{\Rightarrow} \alpha A \delta$ 且 $A \overset{+}{\Rightarrow} \beta$
则称 β 是句型 $\alpha\beta\delta$ 相对于非终结符 A 的 **短语**。
若有 $A \Rightarrow \beta$ 则称 β 是句型 $\alpha\beta\delta$ 相对于该规则 $A \rightarrow \beta$ 的 **直接短语**。
- 一个句型的最左直接短语称为该句型的 **句柄**。

!!注意两个条件
缺一不可

例如
文法:

$$\begin{aligned} E &\longrightarrow T \mid E+T \\ T &\longrightarrow F \mid T * F \\ F &\longrightarrow (E) \mid a \end{aligned}$$

$(a_1 * a_2 + a_3)$ 是文法的句子,
找出此文法的直接短语和句柄。

■ 用子树解释短语，直接短语，句柄:

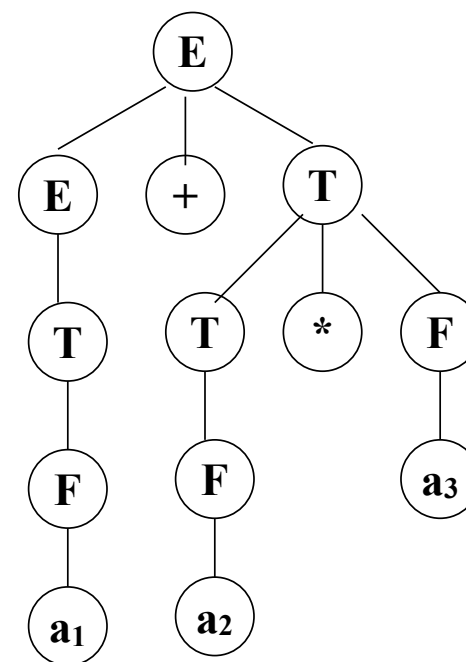
- **短语**: 一棵子树的所有**叶子**自左至右排列起来形成一个相对于**子树根**的短语。
- **直接短语**: 仅有**父子两代**的一棵子树，它的所有叶子自左至右排列起来所形成的符号串。
- **句柄**: 一个句型的分析树中**最左**那棵只有父子两代的子树的所有叶子的自左至右排列。

例子

对表达式文法G[E]和句子a1+a2*a3，挑选出推导过程中产生的句型中的短语，直接短语，句柄。

$$\begin{aligned} E &\longrightarrow T|E+T \\ T &\longrightarrow F|T*F \\ F &\longrightarrow (E)|a \end{aligned}$$

E	短语
$\Rightarrow E+T$	<u>$E+T$</u>
$\Rightarrow E+T*F$	<u>$T*F$</u> , $E+T*F$
$\Rightarrow E+T*a_3$	<u>a_3</u> , $T*a_3$, $E+T*a_3$
$\Rightarrow E+F*a_3$	a_3 , <u>F</u> , $F*a_3$, $E+F*a_3$
$\Rightarrow E+a_2*a_3$	a_3 , <u>a_2</u> , a_2*a_3 , $E+a_2*a_3$
$\Rightarrow T+a_2*a_3$	a_3 , a_2 , <u>T</u> , a_2*a_3 , $T+a_2*a_3$
$\Rightarrow F+a_2*a_3$	a_3 , a_2 , <u>F</u> , a_2*a_3 , $F+a_2*a_3$
$\Rightarrow a_1+a_2*a_3$	<u>a_1</u> , a_2 , a_3 , a_2*a_3 , $a_1+a_2*a_3$

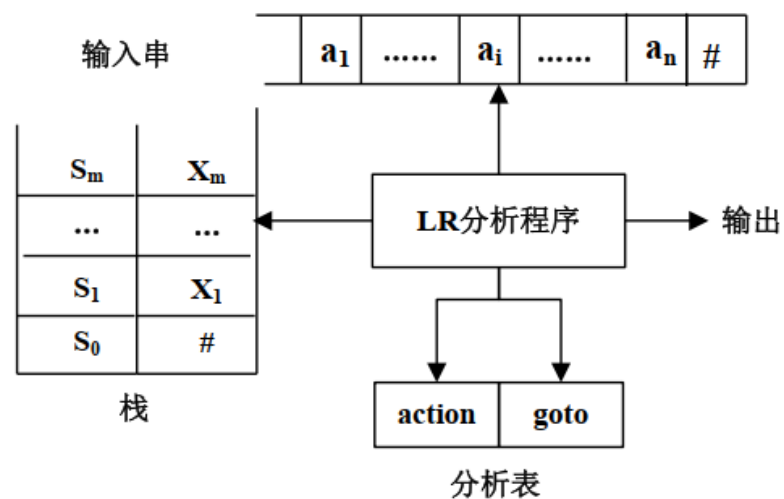


■ LR语法分析器概述

- LR(k)语法分析器适用于一大类上下文无关文法的语法分析。K省略时，假设k是1。
 - L指的是从左向右扫描输入字符串
 - R指的是构造最右推导的逆过程
 - k指的是在决定语法分析动作时需要向前看的符号个数
- 构造LR分析表的三种方法
 - 简单LR方法（SLR），最容易实现，功能最弱
 - 规范LR方法，功能最强，代价最高
 - 向前看的LR方法（LALR），其功能和代价介于前两者之间

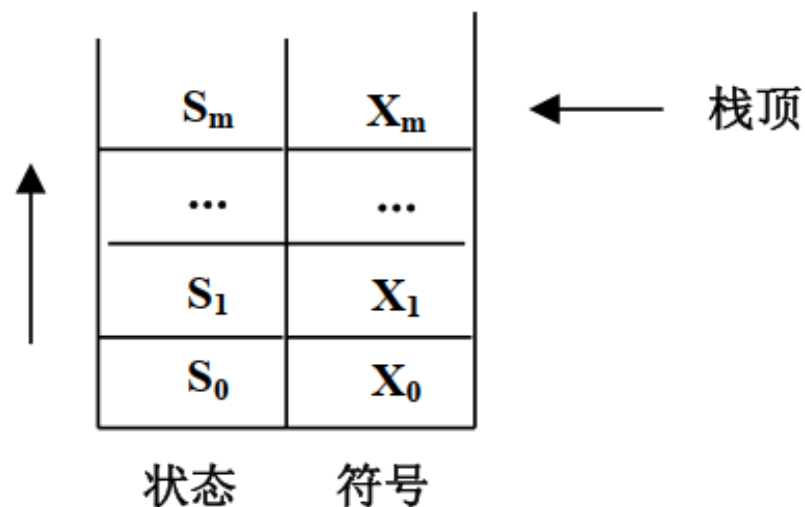
■ LR语法分析器的结构

- 一个LR分析器实质上是一个带先进后出存储器（栈）的确定有限状态自动机(DFA)。



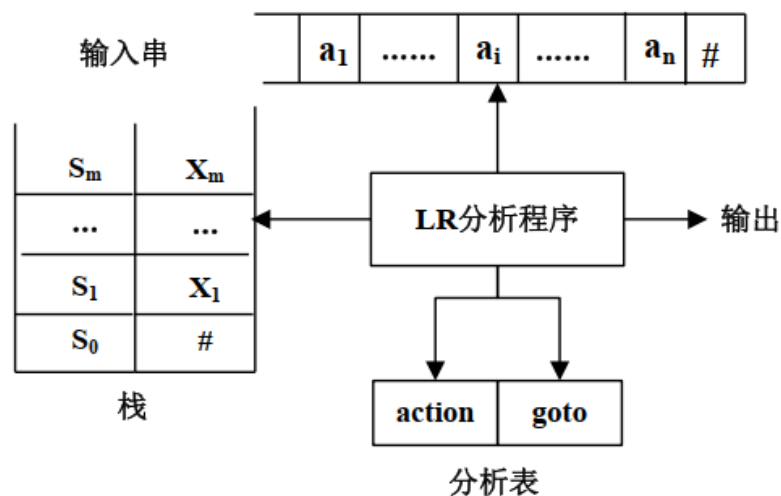
■ LR语法分析器的结构——栈

- 我们将把“历史”和“展望”材料综合的抽象成某些“状态”，存放在栈里。
- 栈里的每个状态概括了从分析开始直到某一归约阶段的全部“历史”和“展望”资料。



LR语法分析器的结构——分析表

- ACTION[s, a]规定了当前状态s面临输入符号a时应采取什么动作。
- GOTO[s, X]规定了状态s面对文法符号X（终结符或非终结符）时下一个状态是什么。



动作表: $\text{action}[S_m, a_i]$ 表示下一个输入符号为 a_i , 栈顶状态为 S_m 时, 分析算法应执行的动作; 若 $\text{action}[s_m, a_i] = s_i$, 表示移进, 然后栈顶状态为 s_i ; 若 $\text{action}[s_m, a_i] = r_j$ 表示使用产生式 (j) 归约; 若 $\text{action}[s_m, a_i] = \text{acc}$ 表示接受输入串; 若 $\text{action}[s_m, a_i] = \text{err}$ 表示语法错误.

状态转换表: $\text{GOTO}[S_i, X]$ 表示归约出非终结符号 X 之后, 当前栈顶状态为 S_i 时, 分析栈应转换到的下一个状态, 即栈顶的新状态.

■ LR语法分析器的结构——ACTION表

- 每一项ACTION[s,a]所规定的动作是下述四种可能之一。
 - **移进**：把(s,a)的下一个状态 $s' = \text{GOTO}[s,a]$ 和输入符号a推进栈，下一个输入符号变成现行输入符号
 - **归约**：指用某一个产生式 $A \rightarrow \beta$ 进行归约。假若 β 的长度为r，归约的动作是A，去除栈顶的r个项，使状态 S_{m-r} 变成栈顶状态，然后把 (S_{m-r}, A) 的下一个状态 $s' = \text{GOTO}[S_{m-r}, A]$ 和文法符号A推进栈。
 - **接受**：宣布分析成功，停止分析器的工作。
 - **报错**：发现源程序含有错误，调用出错处理程序。

注意：若a为终结符，则 $\text{GOTO}[S,a]$ 的值已列在 $\text{action}[S,a]$ 的sj之中(状态j)。因此 GOTO表仅对所有非终结符A列出 $\text{GOTO}[S, A]$ 的值

■ LR分析器的工作原理

分析栈中的串和等待输入的符号串构成如下形式的三元组:

一个LR分析器的工作过程就是
一步一步的变换三元组直至到
“接受”或“报错”为止.

$(S_0S_1S_2\dots S_m, \#X_1X_2\dots X_m, a_ia_{i+1} \dots a_n\#)$

1. 其初态为: $(S_0, \#, a_1a_2 \dots a_ia_{i+1} \dots a_n\#)$
2. 假定当前分析栈的栈顶为状态 S_m ,下一个输入符号为 a_i ,分析器的下一个动作
 - 如果 $\text{action}[S_m, a_i] = \text{移进}$ 且 $S = \text{GOTO}[S_m, a_i]$,则分析器执行移进,三元组变成 $(S_0S_1S_2\dots S_mS, \#X_1X_2\dots X_ma_i, a_{i+1}\dots a_n\#)$ 即分析器将输入符号 a_i 和状态 S 移进栈, a_{i+1} 变成下一个符号
 - 如果 $\text{action}[S_m, a_i] = \text{归约 } A \rightarrow \beta$,则分析器执行归约,三元组变成 $(S_0S_1S_2\dots S_{m-r}S, \#X_1X_2\dots X_{m-r}A, a_ia_{i+1} \dots a_n\#)$,此处 $S = \text{GOTO}[S_{m-r}, A]$, r 为 β 的长度且 $\beta = X_{m-r+1}X_{m-r+2}\dots X_m$.
 - 若 $\text{action}[S_m, a_i] = \text{acc}$,则接收输入符号串,语法分析完成.
 - 若 $\text{action}[S_m, a_i] = \text{err}$,则发现语法错误,调用错误恢复子程序进行处理.

■ LR文法

- 一个文法，如果能为其构造一张分析表,使得它的每个入口均是唯一确定的，则这个文法为**LR文法**。
- 一个文法，如果能用一个每步顶多**向前检查k个输入符号**的LR分析器进行分析，则这个文法就称为**LR(k)文法**。
 - 我们关注 $k \leq 1$ 的情形

LL文法要求每个非终结符的所有候选首字符均不相同，因为预测程序认为，一旦看到首符之后就看准了该用哪一个产生式进行推导。

LR分析程序只有在看到整个右部所推导的东西之后才认为是看准了归约方向。因此，LR方法比LL方法更加一般化。

■ LR(0)项目的定义

- 对于一个文法G，我们首先要构造一个NFA，它能识别G的所有活前缀。这个NFA的每一个状态是下面定义的一个“项目”。
- 文法G的每一个产生式的右部添加一个圆点称为G的一个LR(0)项目（简称项目）。

例如产生式 $A \rightarrow XYZ$ 对应四个项目：

$A \rightarrow \bullet XYZ$

$A \rightarrow X \bullet YZ$

$A \rightarrow XY \bullet Z$

$A \rightarrow XYZ \bullet$

产生式 $A \rightarrow \epsilon$ 只对应一个项目 $A \rightarrow \bullet$ 。

■ LR(0)项目的说明

- 直观上说，一个项目指明了在分析过程的某时刻我们看到产生式多大部分。
 - 例如， $A \rightarrow \bullet XYZ$ 这个项目意味着，我们希望能从后面输入串中看到可以从XYZ推出的符号串。
 - $A \rightarrow X \bullet YZ$ 这个项目意味着，我们已经从输入串中看到能从X推出的符号串，我们希望能进一步看到可以从YZ推出的符号串。
- 将每个项目看成NFA的一个状态，我们就可以构造这样一个NFA，用来识别文法所有的活前缀。

■ LR(0)项目间的转换规则

- 如果状态i和j来自**同一个产生式**，而且状态j的圆点只落后于状态i的圆点一个位置，
 - 如状态i为 $X \rightarrow X_1 \cdots X_{i-1} \bullet X_i \cdots X_n$,
 - 状态j为 $X \rightarrow X_1 \cdots X_i \bullet X_{i+1} \cdots X_n$,
 - 那么就从状态i画一条标志为 X_i 的弧到状态j。
- 假若状态i的圆点之后的那个符号为**非终结符**，
 - 如i为 $X \rightarrow \alpha \bullet A \beta$ ，A为非终结符，
 - 就从状态i画 ε 弧到所有 $A \rightarrow \bullet \gamma$ 状态。
- NFA的接受状态就是那些圆点出现在最右边的项目。

■ LR(0)项目集规范族的构造

- 用 ϵ -CLOSURE(闭包)的办法来构造一个文法G的LR(0)项目集规范族。
- 准备工作：
 - 假定文法G是一个以S为开始符号的文法，我们构造一个文法G'，它包含整个G，但它引进了一个不出现在G中的非终结符S'，并加进一个新产生式 $S' \rightarrow S$ ，而这个S'是G'的开始符号。那么我们称G'是G的拓广文法。这样，便会有一个仅含项目 $S' \rightarrow S \bullet$ 的状态，这就是唯一的“接受”状态。

构成识别一个文法活前缀的DFA的项目集(状态)的全体称为这个文法的LR(0)项目集规范族

■ LR(0)项目集规范族的构造

- 假定 I 是文法 G' 的任一项目集，定义和构造 I 的闭包 $CLOSURE(I)$ 的办法是：
 - I 的任何项目都属于 $CLOSURE(I)$;
 - 若 $A \rightarrow \alpha \bullet B \beta$ 属于 $CLOSURE(I)$ ，那么，对任何关于 B 的产生式 $B \rightarrow \gamma$ ，项目 $B \rightarrow \bullet \gamma$ 也属于 $CLOSURE(I)$;
 - 重复执行上述两步骤直至 $CLOSURE(I)$ 不再增大为止。
- 函数 $GO(I, X)$ 是一个状态转换函数。
 - 第一个变元 I 是一个项目集，
 - 第二个变元 X 是一个文法符号。
 - 函数值 $GO(I, X)$ 定义为 $GO(I, X) = CLOSURE(J)$ ，其中 $J = \{\text{任何形如 } A \rightarrow \alpha X \bullet \beta \text{ 的项目} \mid A \rightarrow \alpha \bullet X \beta \text{ 属于 } I\}$

直观上说，若 I 是对某个活前缀 γ 的有效项目集，那么 $GO(I, X)$ 便是对 γX 有效的项目集。

■ LR(0)文法

- 假如一个文法G的拓广文法G'的活前缀识别自动机的每个状态（项目集）不存在下述情况：
 - 既含移进项目又含归约项目；
 - 含多个归约项目。

则称G是一个LR(0)文法。换言之，LR(0)文法规范族的每个项目集不包含任何冲突项目。

■ LR(0)分析表的构造

- 假定项目集规范族 $C=\{I_0, I_1, \dots, I_n\}$ 。令每一个项目集 I_k 的下标 k 作为分析器的状态。分析表的ACTION子表和GOTO子表可按如下方法构造
 - (0) 令那个包含项目 $S' \rightarrow \bullet S$ 的集合 I_k 的下标 k 为分析器的初态。
 - (1) 若项目 $A \rightarrow \alpha \bullet a \beta$ 属于 I_k 且 $GO(I_k, a) = I_j$, a 为终结符, 置 $ACTION[k, a]$ 为“把 (j, a) 移进栈”, 简记为“ s_j ”。
 - (2) 若项目 $A \rightarrow \alpha \bullet$ 属于 I_k , 对任何终结符 a (或结束符 $\#$), 置 $ACTION[k, a]$ 为“用产生式 $A \rightarrow \alpha$ 进行归约”, 简记为“ r_j ”(假定产生式 $A \rightarrow \alpha$ 是文法 G' 的第 j 个产生式)。
 - (3) 若项目 $S' \rightarrow S \bullet$ 属于 I_k , 则置 $ACTION[k, \#]$ 为“接受”, 简记为“acc”。
 - (4) 若 $GO(I_k, A) = I_j$, A 为非终结符, 则置 $GOTO[k, A] = j$ 。
 - (5) 分析表中凡不能用规则1至4填入信息的空白格均填上“报错标志”。

按上述方法构造的分析表的每个入口都是唯一的。称此分析表是一个**LR(0)**表。使用**LR(0)**表的分析器叫做一个**LR(0)**分析器

■ LR(0)文法的局限性

- 假定一个LR(0)规范族中含有如下的一个项目集（状态）
 $I = \{X \rightarrow \alpha \bullet b \beta, A \rightarrow \alpha \bullet, B \rightarrow \alpha \bullet\}$
- 第一个项目是移进项目，第二、三个项目是归约项目。这三个项目告诉我们应该做的动作各不相同，互相冲突：
 - 第一个项目告诉我们应该把下一个符号b移进；
 - 第二个项目告诉我们应该把栈顶的 α 归约为A；
 - 第三个项目告诉我们应该把栈顶的 α 归约为B。

■ SLR基本算法

- 解决冲突的方法是分析所有含A和B的句型，考察集合FOLLOW(A)和FOLLOW(B)，如果这两个集合不相交，而且也不包含b，那么当状态I面临输入符号a时，我们可以使用如下策略：
 - 若 $a=b$ ，则移进。
 - 若 $a \in \text{FOLLOW}(A)$ ，则用产生式 $A \rightarrow \alpha$ 进行归约；
 - 若 $a \in \text{FOLLOW}(B)$ ，则用产生式 $B \rightarrow \alpha$ 进行归约；
 - 此外，报错

■ SLR基本算法

- 假定LR(0)规范族的一个项目集I中
 - 含有m个移进项目
 - $A_1 \rightarrow \alpha \cdot a_1 \beta_1, A_2 \rightarrow \alpha \cdot a_2 \beta_2, \dots, A_m \rightarrow \alpha \cdot a_m \beta_m;$
 - 同时含有n个归约项目
 - $B_1 \rightarrow \alpha \cdot, B_2 \rightarrow \alpha \cdot, \dots, B_n \rightarrow \alpha \cdot,$
 - 如果集合 $\{a_1, \dots, a_m\}, \text{FOLLOW}(B_1), \dots, \text{FOLLOW}(B_n)$ **两两不相交**（包括不得有两个FOLLOW集合有#），则隐含在I中的动作冲突可以通过检查现行输入符号a属于上述n+1个集合中的哪个集合而解决：
 - 若a是某个 $a_i, i=1,2,\dots,m$ ，则移进。
 - 若 $a \in \text{FOLLOW}(B_i), i=1,2,\dots,m$ ，则用产生式 $B_i \rightarrow \alpha$ 进行归约；
 - 此外，报错

这种冲突的解决方法叫做**SLR(1)**解决办法。

■ SLR语法分析表的构造算法

- 首先把G拓广为G', 对G'构造LR(0)项目集规范族C和活前缀识别自动机的状态转换函数GO。函数ACTION和GOTO可按如下方法构造:
 - 若项目 $A \rightarrow \alpha \bullet a \beta$ 属于 I_k , $GO(I_k, a) = I_j$, a为终结符, 置ACTION[k,a]为“把状态j和符号a移进栈”, 简记为“sj”;
 - 若项目 $A \rightarrow \alpha \bullet$ 属于 I_k , 那么, 对任何非终结符a, $a \in FOLLOW(A)$, 置ACTION[k,a]为“用产生式 $A \rightarrow \alpha$ 进行归约”, 简记为“rj”; 其中, 假定 $A \rightarrow \alpha$ 为文法G'的第j个产生式
 - 若项目 $S' \rightarrow S \bullet$ 属于 I_k , 则置ACTION[k,#]为可“接受”, 简记为“acc”;
 - 若 $GO(I_k, A) = I_j$, A为非终结符, 则置GOTO[k, A]=j;
 - 分析表中凡不能用规则1至4填入信息的空白格均填上“出错标志”。
 - 语法分析器的初始状态是包含 $S' \rightarrow \bullet S$ 的项目集合的状态

按上述方法构造的含有Action和GOTO的分析表, 若每个入口不含多重定义, 则称它为文法G的一张SLR表。具有SLR表的文法G成为一个SLR(1)文法, 使用SLR表的分析器叫SLR分析器。

■ SLR语法分析的局限性

- 所有的SLR语法必须满足如下条件
 - $I = \{X \rightarrow \alpha \cdot b\beta, A \rightarrow \alpha \cdot, B \rightarrow \alpha \cdot\}$
若有: $\text{FOLLOW}(A) \cap \text{FOLLOW}(B) = \emptyset$
 $\text{FOLLOW}(A) \cap \{b\} = \emptyset$
 $\text{FOLLOW}(B) \cap \{b\} = \emptyset$
- 状态I面临某输入符号a
 - 1) 若 $a=b$, 则移进
 - 2) 若 $a \in \text{FOLLOW}(A)$, 则用产生式 $A \rightarrow \gamma$ 进行归约
 - 3) 若 $a \in \text{FOLLOW}(B)$, 则用产生式 $B \rightarrow \delta$ 进行归约
 - 4) 此外, 报错

■ 构造规范LR语法分析表

- 针对SLR语法分析的局限性，我们给出如下解决方案：
 - 重新定义项目，使之包含一个终结符作为第二个分量，可以把更多的信息并入状态中。
 - 项目的一般形式也就变成了 $[A \rightarrow \alpha \bullet \beta, a_1 a_2 \cdots a_k]$ ，其中 $A \rightarrow \alpha \bullet \beta$ 是LR(0)项目，每一个 a 都是终结符或者 $\#$ 。——LR(k)
 - 项目中的 $a_1 a_2 \cdots a_k$ 称为它的**向前搜索字符串**（或展望串）
 - 这样的 a 的集合是FOLLOW(A)的子集，有可能是真子集
 - 归约项目 $[A \rightarrow \alpha \bullet, a_1 a_2 \cdots a_k]$ 意味着：当它所属的状态呈现在栈顶且后续的 k 个输入符号为 $a_1 a_2 \cdots a_k$ 时，才可以把栈顶的 α 归约为 A 。我们只对 $k \leq 1$ 的情形感兴趣 ——LR(1)

■ LR(1)对活前缀有效的定义

- LR(1)的项目 $[A \rightarrow \alpha \cdot \beta, a]$ 对活前缀 γ 有效, 如果存在规范推导 $S \xRightarrow{*} \delta A \omega \Rightarrow \delta \alpha \beta \omega$:
 - $\gamma = \delta \alpha$
 - a 是 ω 的第一个符号, 或者 ω 是空串, a 是 $\#$ 。

• Closure运算的新定义

- 考虑对活前缀 γ 有效的项目集中的项目 $[A \rightarrow \alpha \cdot B \beta, a]$ 必定存在一个最右推导 $S \xRightarrow{*} \delta A \alpha x \Rightarrow \delta \alpha B \beta \alpha x$, 其中 $\gamma = \delta \alpha$
- 假设 $\beta \alpha x$ 能推导出终结字符串 bw , 那么对每一个形如 $B \rightarrow \eta$ 的产生式, 存在推导 $S \xRightarrow{*} \gamma B b w \Rightarrow \gamma \eta b w$, 于是 $[B \rightarrow \cdot \eta, b]$ 对 γ 有效, 其中 b 是 $\text{FIRST}(\beta \alpha x)$ 中的任何终结符。根据 FIRST 的定义, $\text{FIRST}(\beta \alpha x) = \text{FIRST}(\beta a)$

假定 I 是文法 G' 的任一项目集, 构造 I 的闭包 $\text{CLOSURE}(I)$ 的方法是:

- I 的任何项目都属于 $\text{CLOSURE}(I)$;
- 若 $[A \rightarrow \alpha \cdot B \beta, a]$ 属于 $\text{CLOSURE}(I)$, $B \rightarrow \eta$ 是一个产生式, 那么, 对 $\text{FIRST}(\beta a)$ 中的每个终结符 b , 将 $[B \rightarrow \cdot \eta, b]$ 加入 $\text{CLOSURE}(I)$;
- 重复上述两步骤直至 $\text{CLOSURE}(I)$ 不再增大为止。

函数值 $\text{GO}(I, X)$ 定义为 $\text{GO}(I, X) = \text{CLOSURE}(J)$, 其中 $J = \{\text{任何形如 } [A \rightarrow \alpha X \cdot \beta, a] \text{ 的项目} \mid [A \rightarrow \alpha \cdot X \beta, a] \text{ 属于 } I\}$

■ 规范LR语法分析表的构造

• 步骤

- 构造拓广文法 G' 的LR(1)项目集规范族 $C=\{I_0, I_1, \dots, I_n\}$
- 从 I_i 构造语法分析器的状态 i , 状态 i 的分析动作如下:
 - 如果 $[A \rightarrow \alpha \cdot a \beta, b]$ 在 I_k 中, 且 $GO(I_k, a) = I_j$, 则置 $action[k, a]$ 为 s_j , 即“移动(j, a)进栈”, 这里要求 a 必须是终结符
 - 如果 $[A \rightarrow \alpha \cdot, a]$ 在 I_k 中, 且 $A \neq \#$, 则置 $action[k, a]$ 为 r_j , 即按照 r_j 归约, 其中 j 是产生式 $A \rightarrow \alpha$ 的序号
 - 如果 $[S' \rightarrow S \cdot, \#]$ 在 I_i 中, 则置 $action[k, \#]$ 为 acc , 表示接受
 - 状态 k 的转移按照下面的方法确定: 如果 $GO(I_k, A) = I_j$, 那么 $goto[k, A] = j$
 - 其余表项设为出错
- 初始状态是包含 $[S' \rightarrow \cdot S, \#]$ 的项目集构造出的状态。

如果该表无冲突, 那么称该表为 G 的规范LR(1)分析表。具有规范的LR(1)分析表的文法称为一个LR(1)文法。

■ Outline

- 语法制导翻译
 - 属性文法
 - 翻译模式
 - S-属性文法
 - L-属性文法
- 中间代码生成
 - DAG, AST
 - 三地址码
 - 后缀式/逆波兰式

语义分析和中间代码生成

■ 源语言的中间表示方法

- 抽象语法树
- 后缀式
- 三地址代码（包括三元式、四元式、间接三元式）
- 无循环有向图DAG (Directed Acyclic Graph)图表示

■ 语义分析

- 语义分析和中间代码生成是编译程序构造的第三阶段,普遍采用语法制导翻译方法,为每个产生式配一个翻译子程序(称语义动作或语义子程序),并且在语法分析的同时执行这些子程序.
- 语义动作指出:
 - (1)一个产生式所产生的符号的意义;
 - (2) 按照这种意义规定了生成某种中间代码应作哪些基本动作。

■ 属性文法

- **属性文法**：翻译文法的基础上，为每个**文法符号**（终结符或非终结符）配备若干相关的“值”（称为“**属性**”）。
 - 属性代表与文法符号相关的**信息**，如类型、值、代码序列、符号表内容
 - 属性可以**计算和传递**，属性加工过程即是**语义处理过程**
 - 语法树结点中的属性要用**语义规则**来定义,语义规则和相应结点的产生式相关

属性(attribute)：编程语言结构的任意特性

- 静态(static)：执行之前绑定的属性
- 动态(dynamic)：执行期间绑定的属性
- 属性的典型例子有：变量的数据类型：静态；表达式的值：动态；存储器中变量的位置：静态或动态；程序的目标代码：静态

- **语义规则**：文法的每个产生式都配备一组属性的**计算规则**
 - 包括属性计算，静态语义检查，符号表操作，代码生成等

■ 属性的计算

- 终结符只有综合属性，它们由词法分析器提供；
- 非终结符既可有综合属性也可有继承属性，文法开始符号的所有继承属性作为属性计算前的初始值。

- 属性：综合属性与继承属性
 - 综合属性用于“自下而上”传递信息（用 $\uparrow c$ 表示，其中 $\uparrow$ 为综合属性标记， C 为属性值）
 - 继承属性用于“自上而下”传递信息（用 $\downarrow c$ 表示，其中 $\downarrow$ 为继承属性标记， C 为属性值）
- 属性文法中，对应于每个产生式 $A \rightarrow \alpha$ 都有一套与之相关联的语义规则，每条规则的形式为 $b := f(c_1, c_2 \dots c_k)$ ， f 是一个函数， b 和 $c_1, c_2 \dots c_k$ 是该产生式文法符号的属性。并且
 - b 是 A 的一个综合属性，且 $c_1, c_2, \dots, c_k$ 是该产生式右边文法符号的属性
 - b 是产生式右部某个文法符号的一个继承属性，且 $c_1, c_2, \dots, c_k$ 是 A 或者产生式右边任何文法符号的属性
 - 在这两种情况下，我们说属性 b 依赖于 $c_1, c_2, \dots, c_k$ 。

例： $A \rightarrow X_1 X_2 \dots X_n$

A 的综合属性 $S(A)$ 计算公式：

$S(A) := f(I(X_1), \dots, I(X_n))$

X_j 的继承属性 $T(X_j)$ 计算公式：

$T(X_j) := f(I(A), \dots, I(X_n))$

■ L属性文法定义

- 在语法分析过程中进行翻译时，属性的计算顺序将与分析方法建立分析树节点的顺序相关。有一种能够描述许多自顶向下和自底向上翻译方法的自然顺序——**深度优先顺序**。
- L属性定义
 - 一个属性文法是**L-属性文法**，如果对于每个产生式 $A \rightarrow X_1 X_2 \cdots X_n$ ，其每个语义规则中的每个属性，或者是综合属性，或者是 X_j ($1 \leq j \leq n$) 的一个继承属性且这个继承属性仅依赖于：
 - (1) 产生式 X_j 在左边符号 $X_1, X_2, \dots, X_{j-1}$ 的属性；
 - (2) A 的**继承属性**。
- L属性的计算
 - 可以使用深度优先顺序来计算
 - LL(1)的分析过程，可以看成是深度优先建立语法树的过程，可以在自上而下语法分析的同时实现L属性文法的计算

S-属性文法一定是L-属性文法

■ S-属性文法的特点

- S-属性文法，只有综合属性
- 产生式左边的文法的综合属性要根据产生式右边符号的综合属性来进行计算。
- 适用于那些需要类似于表达式，需要计算结果的文法。
- 综合属性可以在分析输入符号串的同时由自底向上的分析器来计算。
 - 分析器保存与栈中文法符号有关的综合属性
 - 每当归约时，新的属性值就由栈中正在归约的产生式右边符号的属性值来计算。

运行时环境

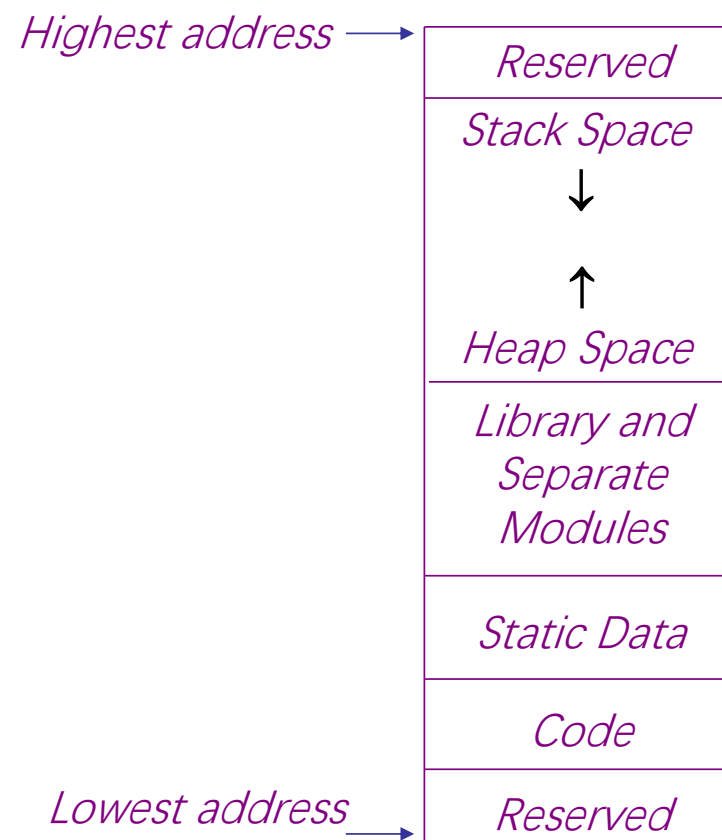
■ 运行时环境

- 在生成目标代码之前，需要把静态的程序和实现这个程序的运行时的活动联系起来，主要是存储组织与管理
 - 活动记录的建立与管理
 - 存储器的组织与存储分配策略
 - 非局部名称的访问

■ 程序运行时存储空间的布局 (*layout*)

– 典型的程序布局

- 保留地址区
目标机体系结构和操作系统专用
- 代码区 静态存放目标代码
- 静态数据区
静态存放全局数据
- 共享库和分别编译模块区
静态存放这些模块的代码和全局数据
- 动态数据区
运行时动态变化的堆区和栈区



■ 存储分配策略

- 存储分配的三种策略

- 静态分配策略

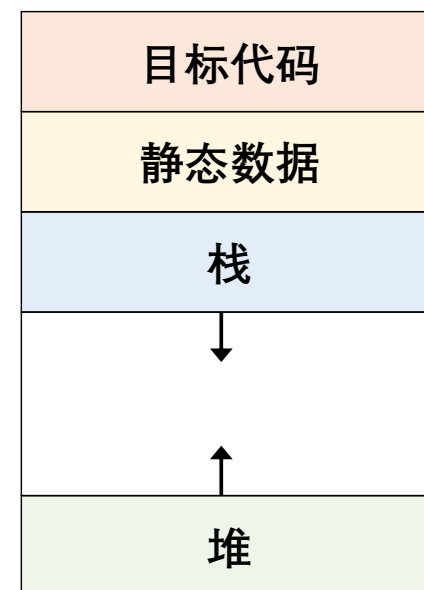
- 在编译时为所有数据对象分配固定的存储单元，且在运行时始终保持不变

- 栈式动态分配策略

- 在运行时按栈方式管理运行时的存储空间

- 堆式动态分配策略

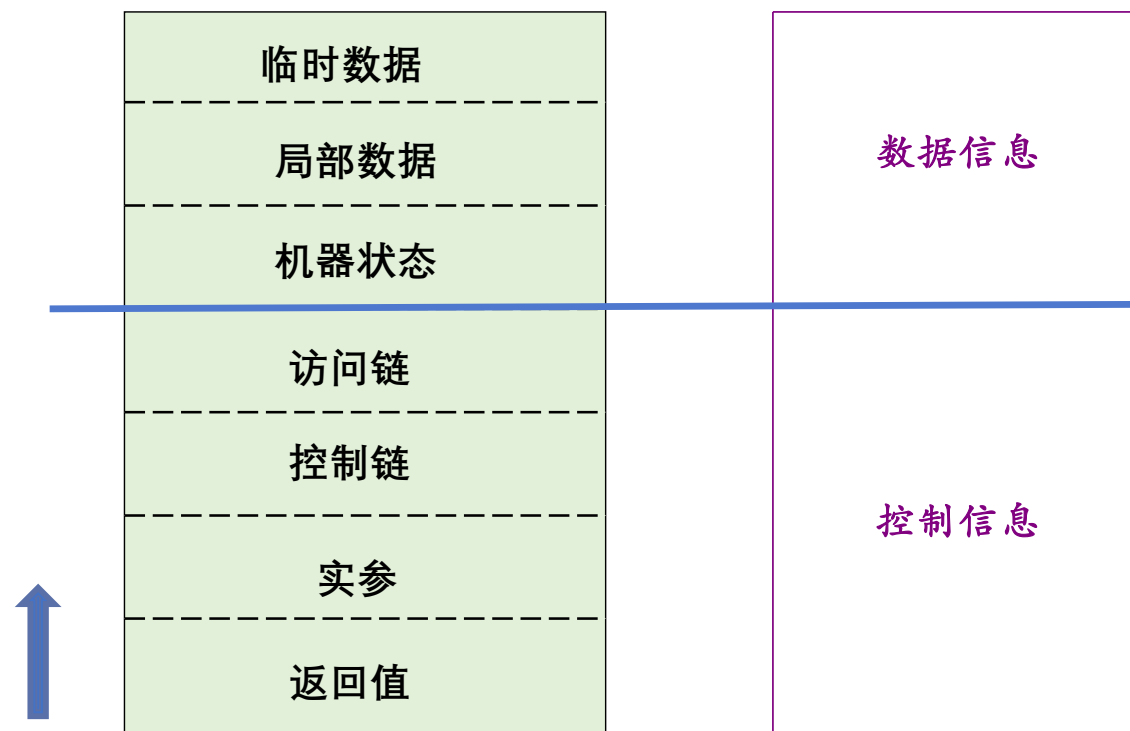
- 在运行时根据需从堆数据区域分配和释放存储空间



■ 活动记录

• 活动记录的定义

- 过程一次执行所需要的信息用一块连续的存储区来管理，这块存储区叫做活动记录
- 在过程调用时将活动记录压入栈，在控制返回调用者时把活动记录弹出栈



优化

■ 问题的引入

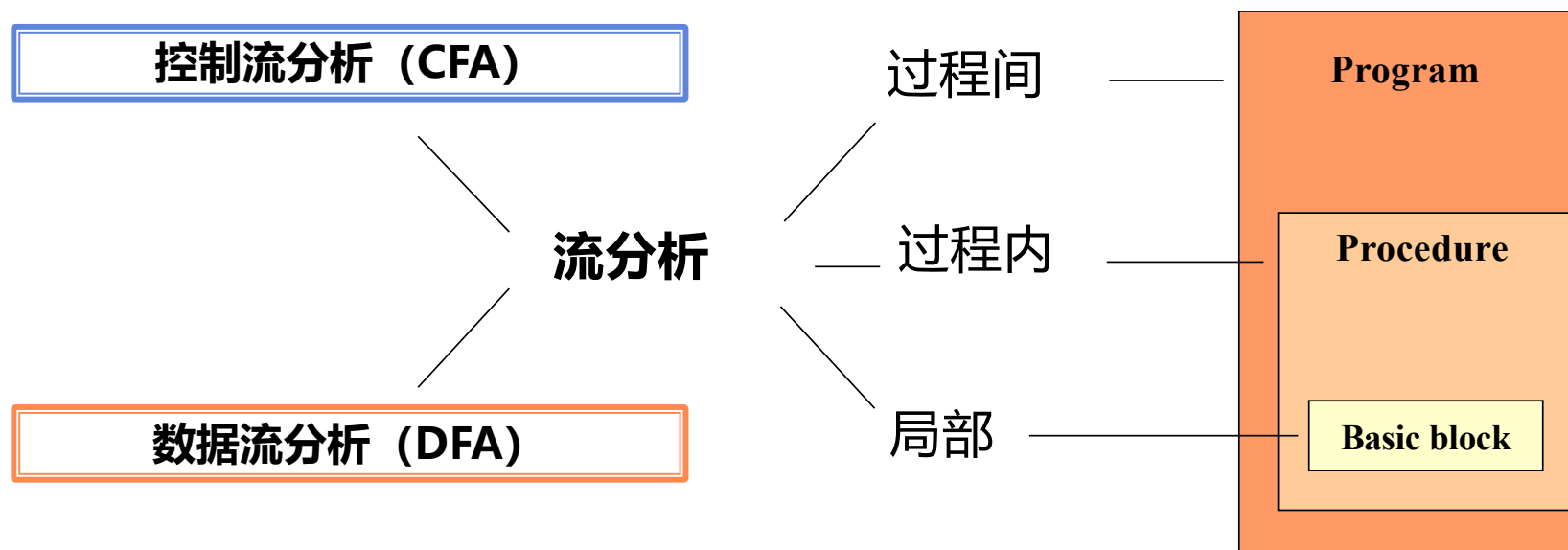
- 优化 = 分析 + 转换
- 分析是优化的基础
 - 流分析
 - 控制流分析 (Control flow Analysis)
 - 数据流分析 (Data flow Analysis)
 - 别名分析 (Alias Analysis)
 - 依赖关系分析 (Dependence Analysis)

本章目标：理解和掌握控制流分析和数据流分析的基本原理和方法

一、控制流分析

➤ 控制流分析

✦ 分析判断程序的控制结构，构建程序的控制流图 (*Control Flow Graph, CFG*)



✦ 控制流图是后续程序分析和变换的基础

■ 1、基本块和控制流图

- 基本块(Basic Block)
 - 基本块是一段只能从它的开始处进入，结束处离开的顺序代码序列
 - 基本块只有最后一条语句是分支语句，并且只有第一条语句是分支的目标
 - 在基本块内部，除了第一条指令外，每条指令都只有一个前驱；除了最后一条指令外，每条指令只有一个后继
- 使用基本块代替指令，可以减少分析的时间和空间开销

■ 1.1 识别基本块

1. 首先，扫描程序，使用下列规则识别基本块的首语句：

(规则1) 程序/函数的**第一条语句**

(规则2) 分支语句的**目标** (对大多数中间代码，
分支的目标是带有标号的语句)

(规则3) 任何**紧跟**在分支或者return语句后面的语句

2. 每个首语句对应的基本块包括了首语句+下一个首语句之间（不包括下一个首语句）的语句

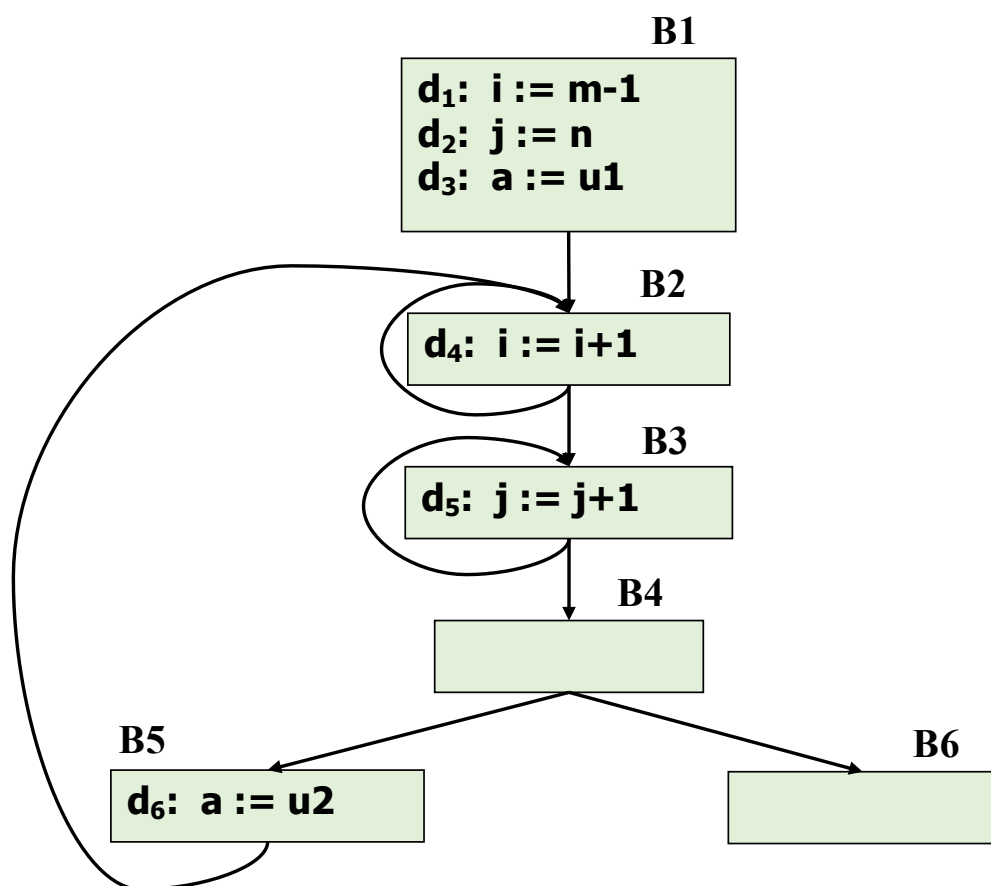
■ 1.2 控制流图

- 控制流图 (*control flow graph*, CFG) 是一个有向流图 $G=(N,E)$, 其中:
 - 节点 N 表示基本块
 - 边 E 表示程序的控制流向
- 首语句是第一条语句的称为开始节点 (*start node*)
- CFG中的边只表示程序可能走这条路径

■ 1.2 构建控制流图

- 如果基本块BB1和BB2满足下面的条件之一，则存在一条从BB1到BB2的有向边
 - (1) 从BB1的最后一条语句可以跳转到BB2的第一条语句
 - (2) BB2紧跟在BB1之后，并且BB1的**最后一条指令不是无条件转移**

1.4 点 (Point) 和路径 (Path)

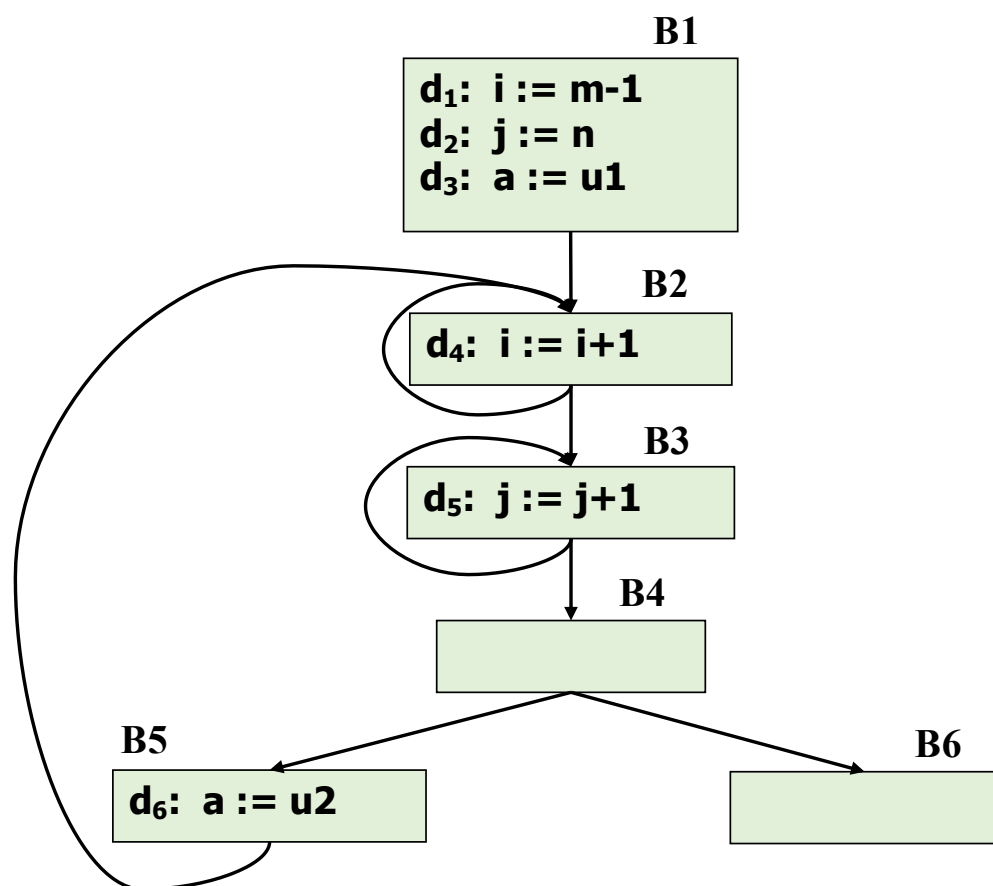


一个基本块内的点:

- 语句之间
- 第一条语句之前
- 最后一条语句之后

第一条语句之前: 基本块入口点
最后一条语句之后: 基本块出口点

1.4 点 (Point) 和路径 (Path)



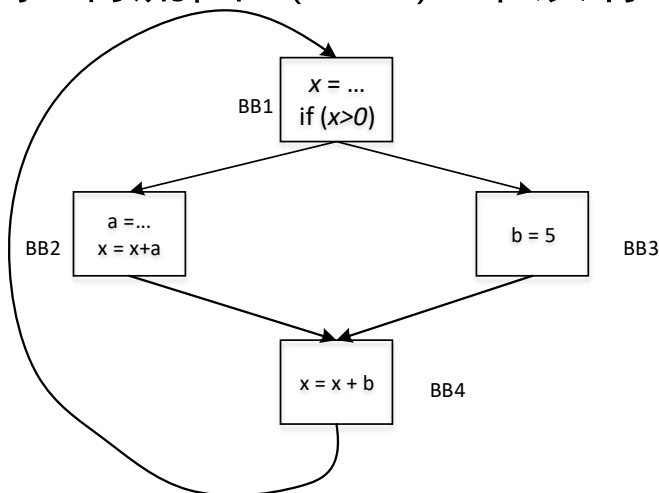
路径是点 $p_1, p_2, \dots, p_n$ 组成的序列,

满足:

- (i) 如果 p_i 紧邻在语句 S 之前, p_{i+1} 紧跟在 S 之后
- (ii) 或者 p_i 是一个基本块的出口点, p_{i+1} 是后续基本块的入口点

二、数据流分析

- 控制流图（CFG）中没有包含程序数据信息

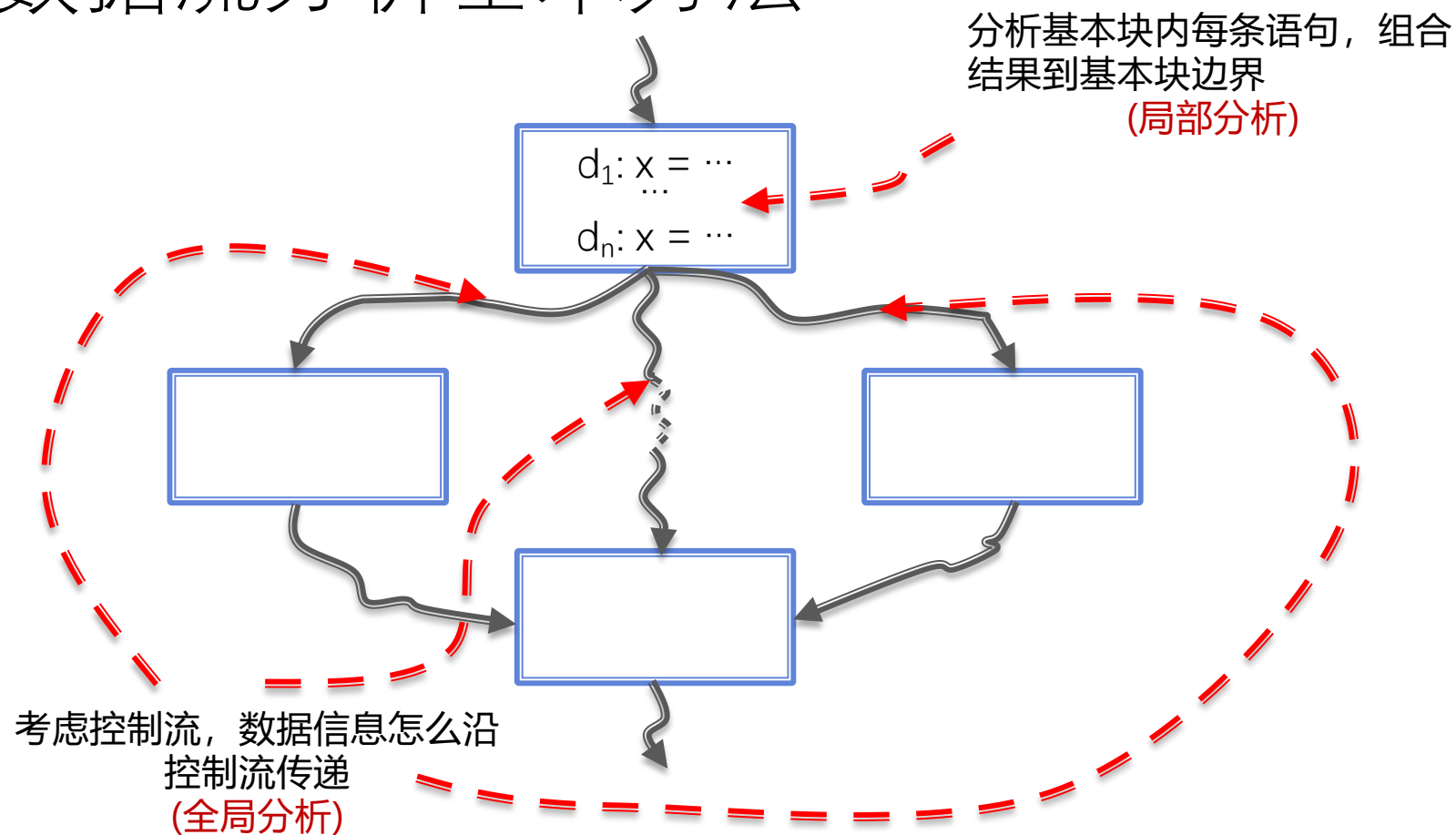


□基本块BB2定值了变量a，除了BB2，a是否还会被其它基本块使用？

□ 在BB4入口b是否是一个常数？

- 数据流分析（Data Flow Analysis, DFA）
 - 获取和分析程序中数据的相关信息，获取数据流信息

2.1 数据流分析基本方法



■ 2.2.1 到达-定值

- 到达-定值 (Reaching Definitions)
 - 变量 x 有一次定值 d ，如果存在一条从定值点开始的路径到达点 p ，并且沿着这条路径，定值 d 没有被其他定值杀死，那么就说定值 d 到达点 p
- 到达-定值分析
 - 对每个基本块，分析求解到达基本块边界（入口点和出口点）的所有定值

■ 2.2.2 到达-定值分析方法

$$\text{in}[B] = \bigcup_{P_i \in \text{pred}[B_n]} \text{out}[P_i]$$

$$\text{out}[B] = \text{gen}[B] \cup (\text{in}[B] - \text{kill}[B])$$

控制流图中，特殊的两个空结点：Entry、Exit

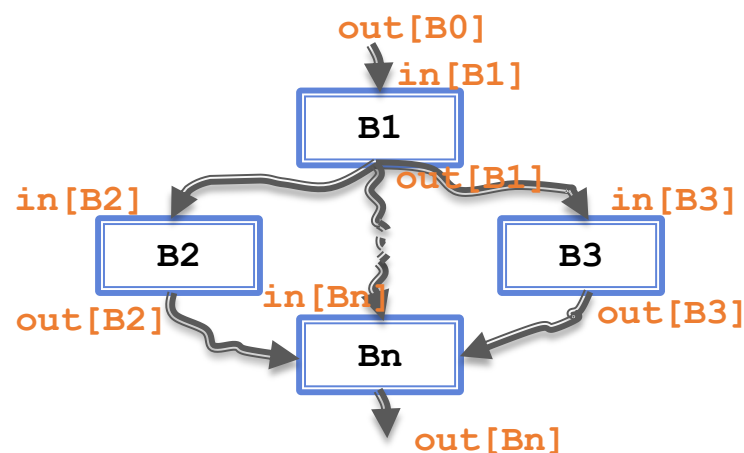
显然有， $\text{out}[\text{Entry}] = \emptyset$

求解方程组方法：不动点方法

■ 3 活跃变量

- 如果一个变量 v 在点 p 开始的某条路径上使用, 那么变量 v 在点 p 活跃 (*live*)
- 否则, 变量 v 在点 p 是死变量
- 活跃变量分析
 - 对每个基本块 B , 分析获取 B 边界处 (入口和出口点) 活跃变量集合

3.2 活跃变量分析方法



```
out[B2]=in[Bn]
in[B2]=use[B2] U (out[B2]-def[B2] )
```

```
out[B1]=in[B1] U in[B2] U ... =  $\bigcup_{Si \in succ[B1]} in[Si]$ 
in[B1]=use[B1] U (out[B1]-def[B1])
```

■ 3.4 总结：前向分析和后向分析

- **前向分析**：在点p计算过去的行为产生的数据流信息
 - 计算CFG中前驱
 - 例如：到达-定值分析
- **后向分析**：在点p计算未来行为的数据流信息
 - 计算 CFG中的后继
 - 例如：活跃变量分析

■ 优化

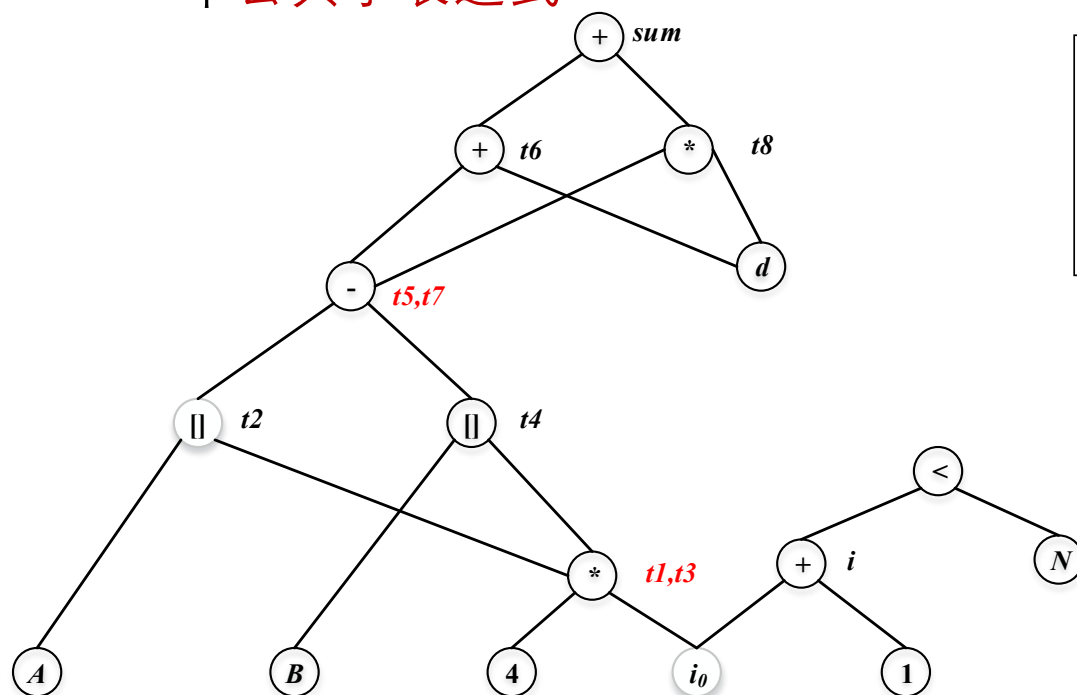
- 优化的原则
 - 安全原则，代码变换必须是等价的
 - 代码变换应该为大多数程序生成“更好的”代码，是有利可图的
 - 代码变换是代价适宜的
 - 优化时间
 - 所需要的（内存）资源
 - 实现优化的复杂度

■ 一、基本块优化

- 利用有向无环图（DAG）实现基本块内的优化
 - 基本块的DAG是节点带有如下标记的有向无环图，能够清晰地展示基本块内每条语句计算的值如何在基本块内被使用
 1. 叶节点(没有后继的节点)：以唯一标识符(变量名)或常数作为标记，表示该节点代表该变量或常数的值。如果叶节点用来代表某变量A的地址，则用addr(A)作为该节点的标记。有时候，我们还会把叶节点上作为标记的标识符加上下标0，以表示它是该变量的初值。
 2. 内部节点(有后继的节点)：以操作符作为标记，表示该节点代表应用该操作符对其后继节点所代表的值进行运算的结果。
 - 节点上可能附加一个或多个标识符，表示这些变量具有该节点所代表的值

2、基于DAG优化

- 公用子表达式删除 (Common Subexpression Elimination, CSE)
 - 如果DAG某内部节点上附有多个标识符, 则表示计算该节点值的表达式是一个公共子表达式



2 $t2 = A[t1]$
3 $t3 = 4*i$
4 $t4 = B[t3]$



2 $t2 = A[t1]$
3 $t3 = t1$
4 $t4 = B[t3]$

■ 二、全局优化

- 全局优化作用于整个过程（函数），进行跨基本块的优化
- 为了保证优化的正确性，全局优化通常先进行数据流分析，利用数据流分析的结果进行代码变换

■ 1. 循环不变量外提

- 如果循环中的一个计算在循环的每次迭代都产生相同的值，则该计算被称作循环不变量。
- 当循环的基本块的一个操作的操作数都满足下列条件时，该操作是循环不变的：
 - 操作数是常数，或者
 - 所有到达操作数的定值都在循环之外，或者
 - 只存在一个到达操作数的定值，并且该定值在循环内，并且该定值本身是循环不变的

■ 2.归纳变量与强度削弱

- 归纳变量 (Induction Variable)

- 在循环中，如果变量x的值每次改变都是增加或者减少某个固定的常数，那么x就是循环的归纳变量

```
for (i=0;i<N; i++){  
    j = 2*i +1;  
    a[i]=j  
}
```



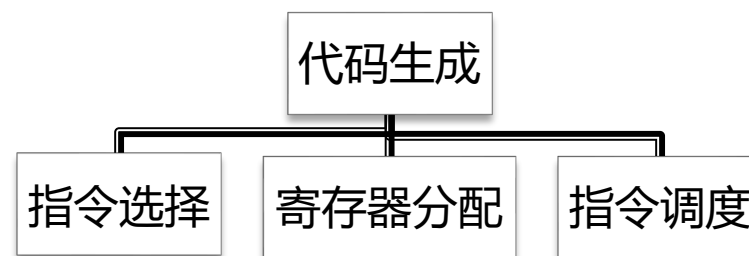
```
for (i=0;i<N; i++){  
    j += 2;  
    a[i]=j  
}
```

- 循环控制变量i每次递增1
- $j=2*i+1$ ，每次递增2

强度削弱

目标代码生成

■ 目标代码生成涉及的问题



- 指令选择
 - 将中间代码翻译成等价的 **目标机指令集指令序列**，实现中间代码到目标机指令的映射
- 寄存器分配
 - 对于load/store架构的目标机是必须要考虑的问题
- 指令调度
 - 通过重排指令，减少流水线中的停顿，最大化指令级并行

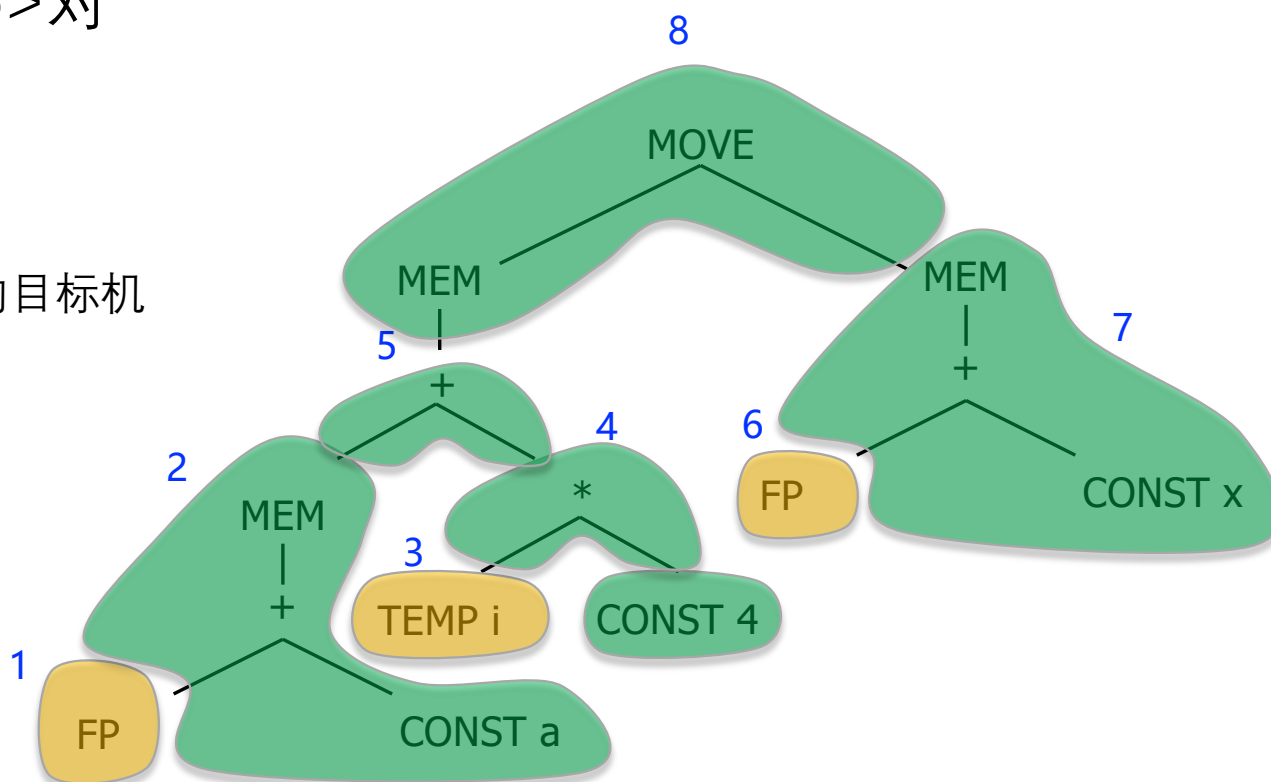
■ 1.3 指令选择的基本思想

- 模式匹配(Pattern-Match)进行指令选择
 - 基于模式匹配技术选择与一段IR匹配的指令
 - 直到获得覆盖全部IR的指令序列
- 指令选择方法包括
 - 基于树模式匹配的指令选择
 - 基于窥孔匹配的指令选择
- 指令选择阶段假设有足够多的符号寄存器
 - 到寄存器分配阶段再考虑符号寄存器到物理寄存器的分配

3.3 平铺方案

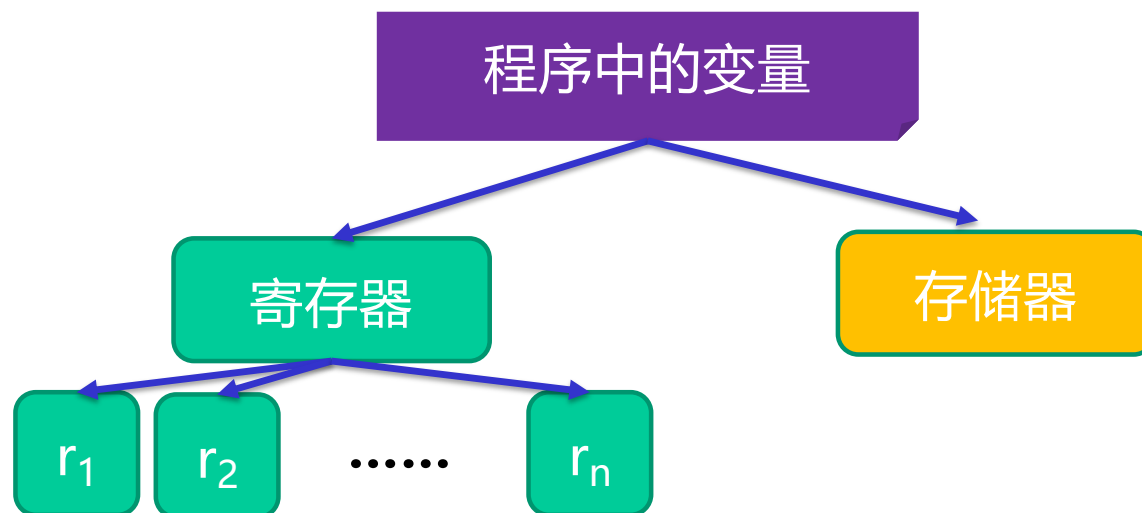
- 平铺方案是一组<node,op>对
 - node是IR树中的一个节点
 - op是一个操作树 (瓦片)
 - 一个<node,op>对表示op对应的目标机指令可以覆盖node为根的子树

• $a[i]=x$ 平铺方案



■ 1.1 什么是寄存器分配

- 将程序中的变量分配到寄存器或者存储器的过程，寄存器分配确定
 - 寄存器分配：哪些变量保存在寄存器中，哪些变量保存在存储器中
 - 寄存器指派：对于保存在寄存器中的变量：具体放在哪个寄存器



■ 1.2 寄存器分配的原则

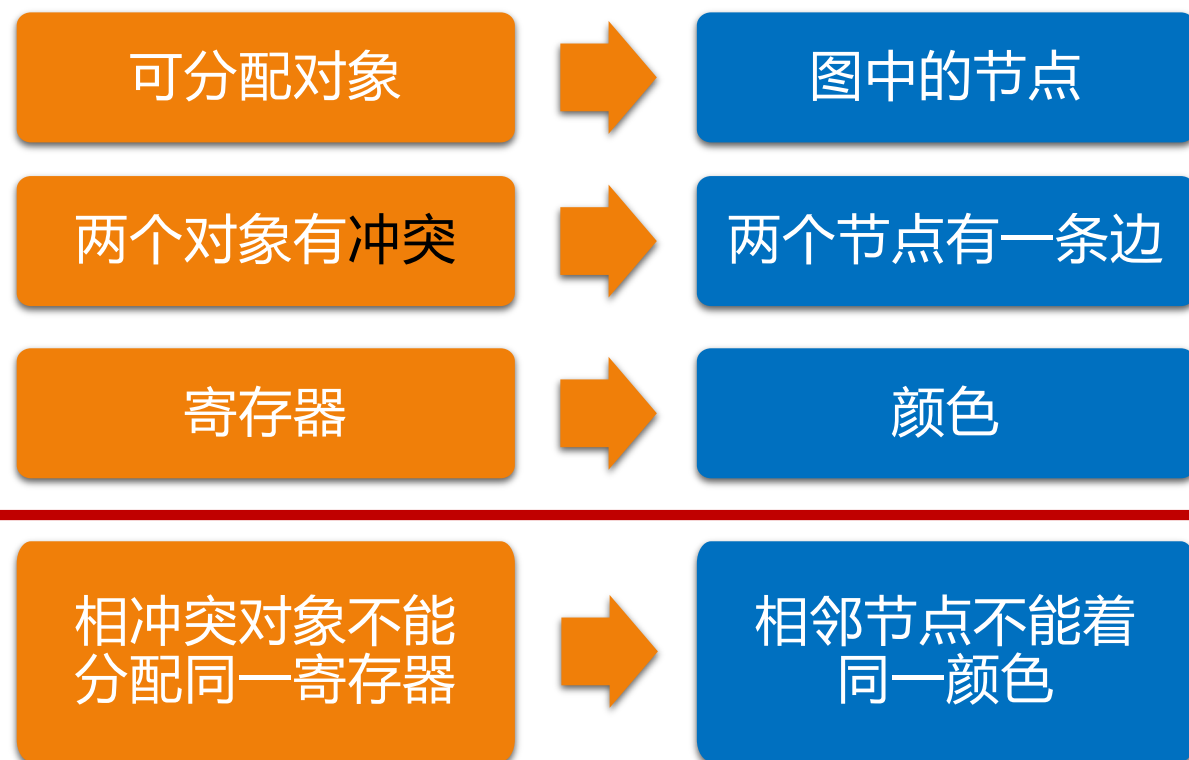
- 程序变量数目 vs. 寄存器数目
 - 程序中有大量的变量，变量数一般大于寄存器数
- 溢出
 - 将一个变量从寄存器中送回到存储器中的动作叫做寄存器溢出 (Spill)
 - 溢出意味着插入一条写指令，并且下次使用该变量时，需要再为该变量分配寄存器，并且通过读指令将值从存储器中读入到指派的寄存器
 - 尽可能减少溢出

■ 1.2 寄存器分配的原则

- 寄存器分配的原则
 - 尽可能地将较多的变量保存在寄存器中
 - 尽可能地将频繁使用的变量保存在寄存器中
(将那些使用较少的变量存放在存储器中)
- 一个好的编译器应设计出尽可能满足这两个标准的寄存器分配算法

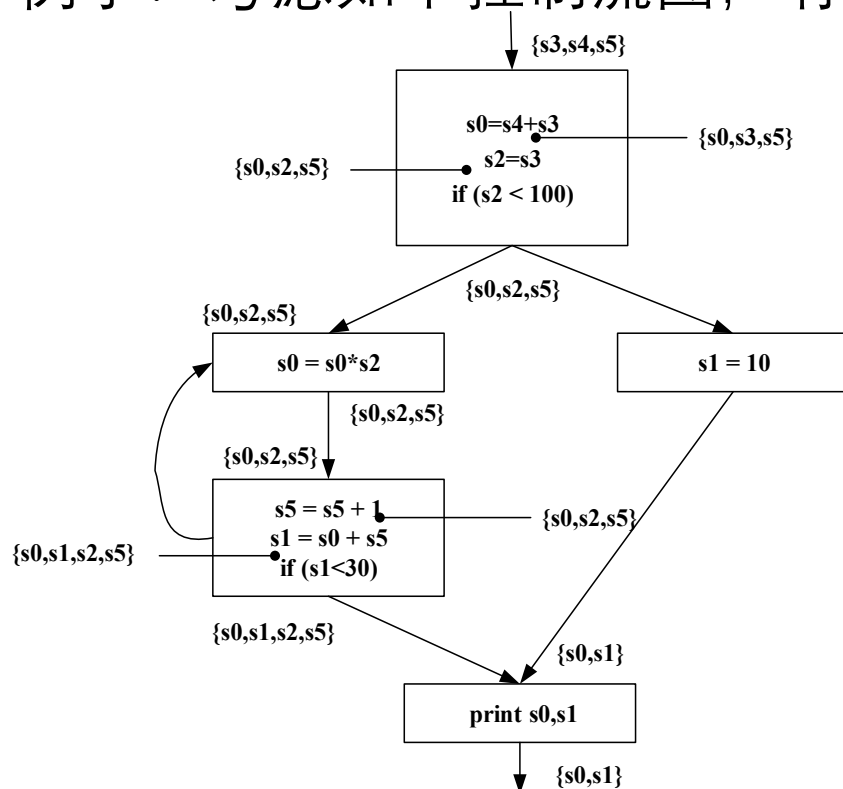
3.2 寄存器分配与图着色的关联

寄存器分配 图着色



3.4.1 冲突图的构建

- 例子：考虑如下控制流图，有s0~s5共6个符号寄存器



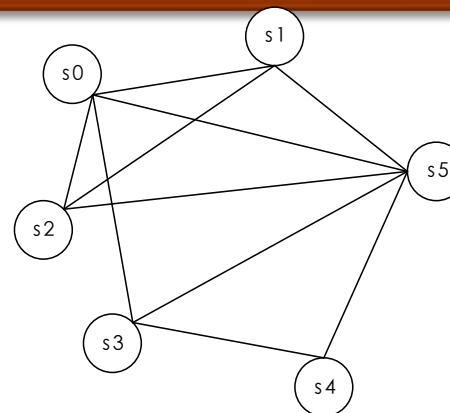
确定可分配对象



确定程序中每一个点的活跃变量集合



确定冲突图的边



■ 4.2 通过化简着色

- 对于 k -可着色图，通过化简着色包括2个步骤

化简冲突图

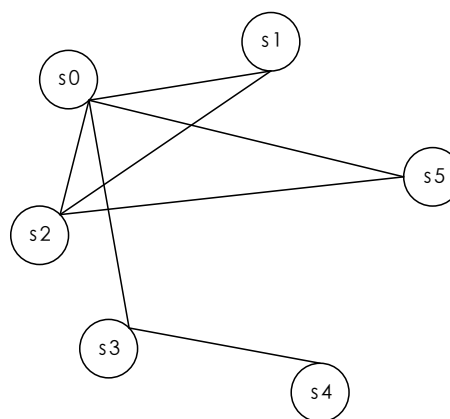
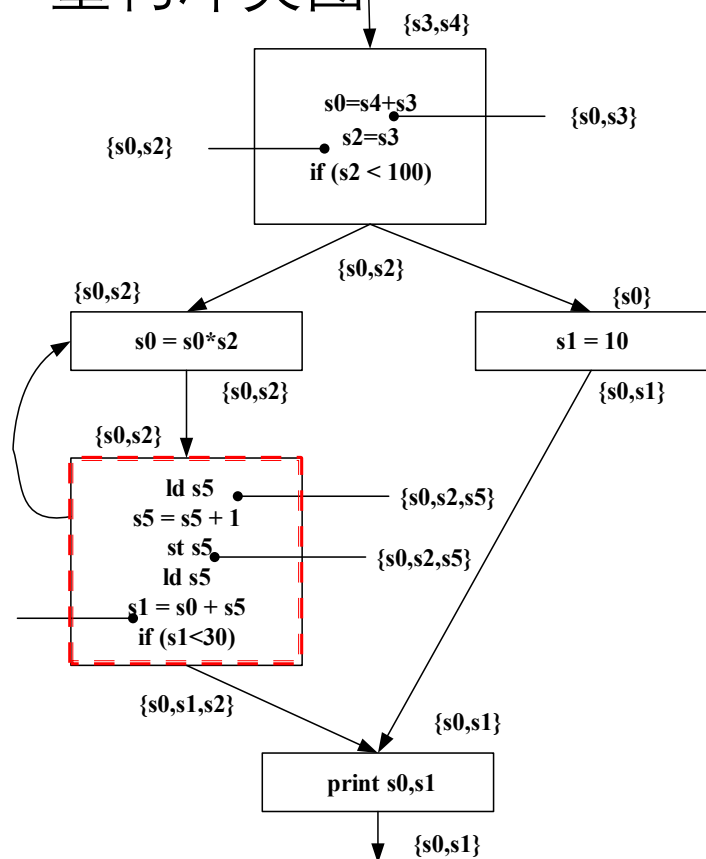


选择颜色

- 如果一个冲突图是 k -可着色的，则存在一个使用不超过 k 种颜色的寄存器分配方案
- 对于寄存器分配问题，不能保证一个冲突图一定是 k -可着色的
- 如果冲突图不是 k -可着色的，怎么办？

4.3 溢出

• 重构冲突图



□ 溢出的作用是减少图中的冲突边，以减少寄存器的压力

□ 溢出代价很大，是寄存器分配需要极力避免的

可以的化简顺序：S4、S3、S5、S1、S0、S2

是否还有其它方法可以减少寄存器的压力，从而避免溢出呢？

■ 4.4 合并

- 考虑如下复制指令，x赋值给y后不再使用x

- x和y不同时活跃 (没有冲突边)
- x和y可以使用同一个寄存器
- 从而可以删除复制指令

```
x ← ...  
...  
y ← x (之后不再使用x)  
...  
... ← y
```

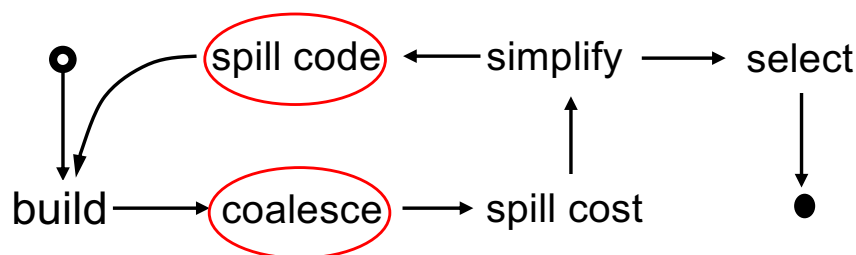
- 合并**(coalescing):寻找可以删除的复制指令并合并其节点的过程
 - 如果x和y不冲突，且由一条复制指令相关联，则可将x和y合并成一个节点，使x和y使用同一个寄存器(减少了冲突图的节点数)

■ 4.4 合并

- 合并
 - 通过减少节点数减少寄存器分配的压力
 - 活跃区间不会发生变化, 无需重构冲突图
 - 合并后的新节点的边是原来两个节点的边的并集
- 溢出
 - 通过减少冲突边减少寄存器分配的压力
 - 活跃区间发生变化, 重构冲突图

■ 4.5 Chaitin算法

- 构建冲突图、合并节点、化简冲突图、溢出节点的迭代过程



- 奠定了图着色寄存器分配算法的基础
- 后续改进算法主要在合并和溢出部分

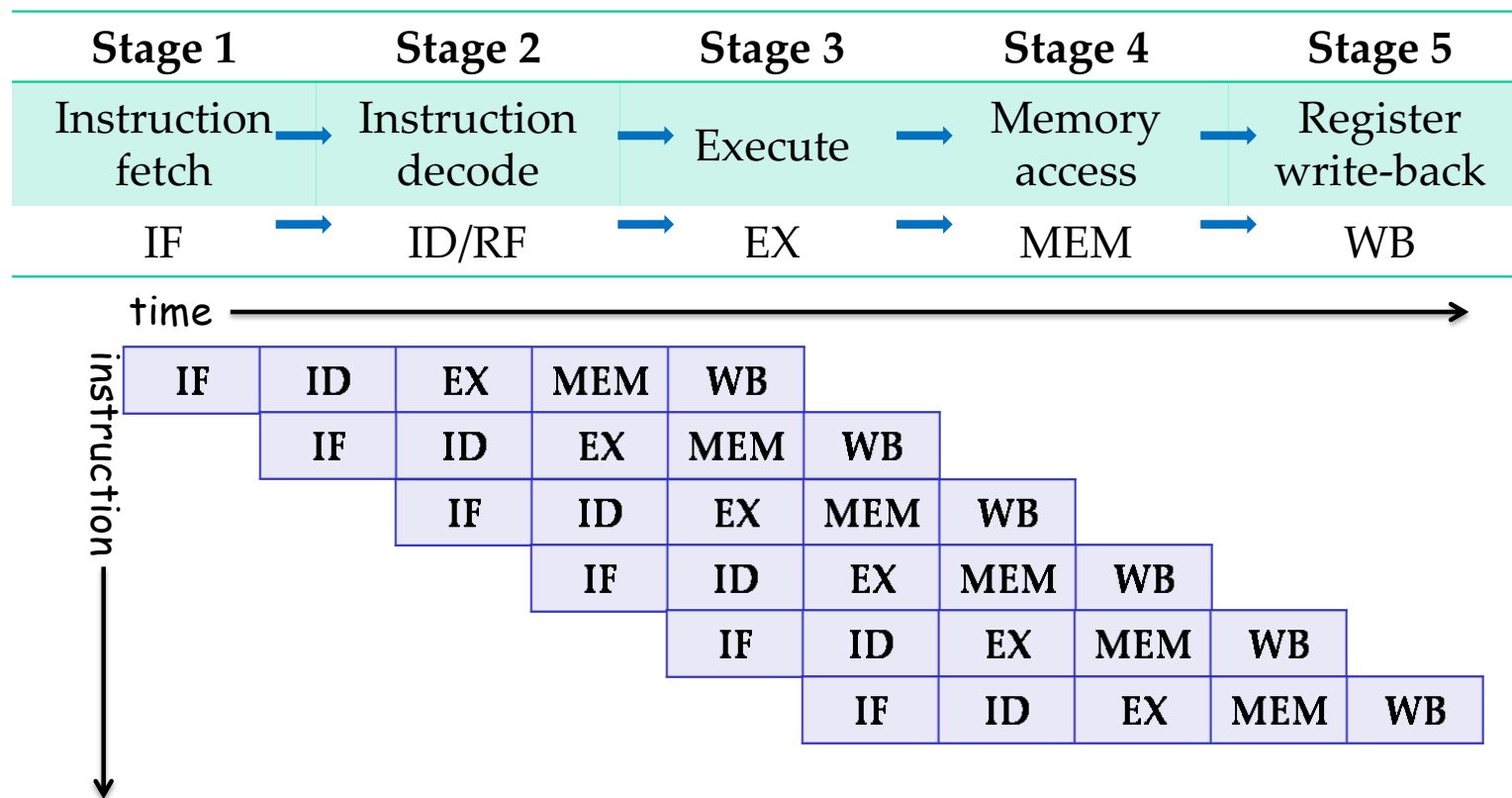
■ 五、指令调度

- 指令序列中各操作的**执行顺序**影响总执行时间
- 指令调度通过**重排**指令序列中各**操作的执行顺序**，试图减少总执行时间



- 指令调度的目标
 - 最大化指令级并行
 - 减少流水线中的**气泡**(bubbles)
- 指令调度需要考虑的问题非常多，一个最优的指令调度是一个NP完全问题

1.1 流水线



如何调度一个程序的指令，才能提高指令级并行的能力？

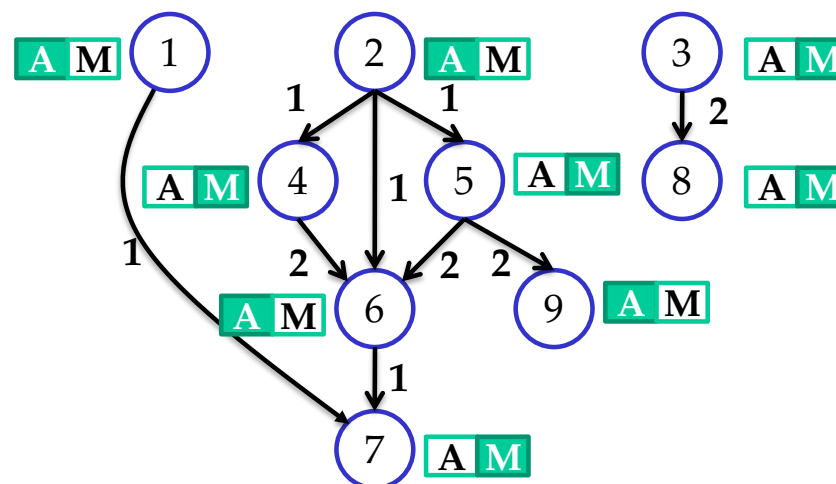
■ 1.3 硬件如何解决指令调度的限制

- 结构冲突
 - 复制多个功能部件 (expensive)
 - 更深的流水
- 控制冲突
 - 硬件分支预测 (branch prediction)
 - 运行时投机执行 (runtime speculation)
- 数据冲突
 - 数据定向技术 (data forwarding)
 - 运行时投机执行

3.2 带延迟的数据依赖图

• 例子

1	addi t2, t1, 1
2	addi sp, sp, 12
3	sw t0, 0(t6)
4	lw t3, -4(sp)
5	lw t4, -8(sp)
6	addi sp, sp, 8
7	sw t2, 0(sp)
8	lw t5, 0(t6)
9	addi t4, t4, 1



假设：

latency (ALU op) = 1

latency (MEM op) = 2

■ 期末考试

- 时间：2026年6月18日 8:00
- 地点：教学三楼3102
- 形式：闭卷
- 题型：选择、简答、推导计算等，与平时作业、测试类似
- 要求：提前十分钟到

遵守中国人民大学考试纪律！！

■ Last Words

Thank you!

Take Care!

Good Luck!